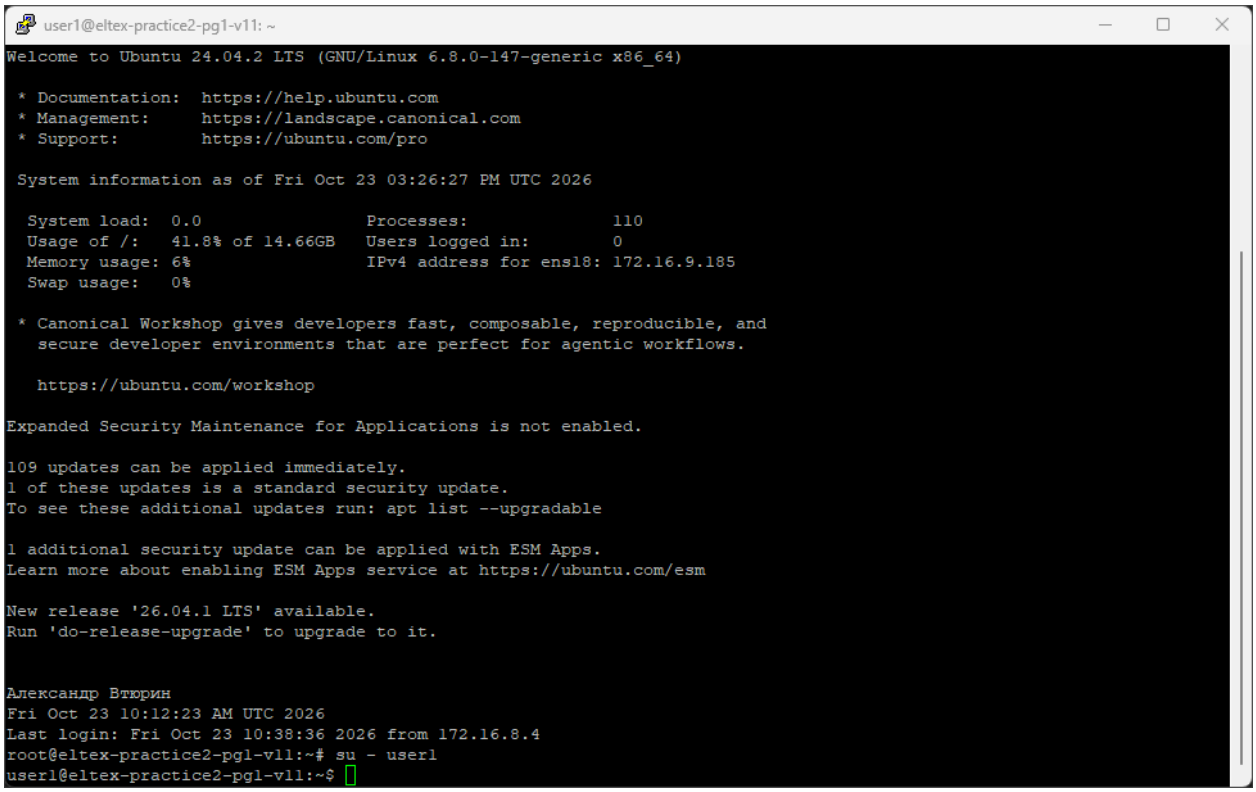
Задания. Раздел 1.

1) Войдите под пользователем user1 из практики 2 (su - user1)
2) Подсчитайте количество процессов, имеющих несколько потоков выполнения
3) Запустите top и настройте вывод полей с информацией о процессе следующим образом:
   • отсортируйте список процессов по убыванию значения поля VIRT, зафиксируйте результат скриншотом, повторите для полей RES и SHR, в отчете поясните, что описывают данные показатели;
   • удалите поля VIRT, RES, SHR и добавьте поле RUSER, сделайте так, чтобы это поле было показано после поля USER, поясните отличие RUSER/USER;

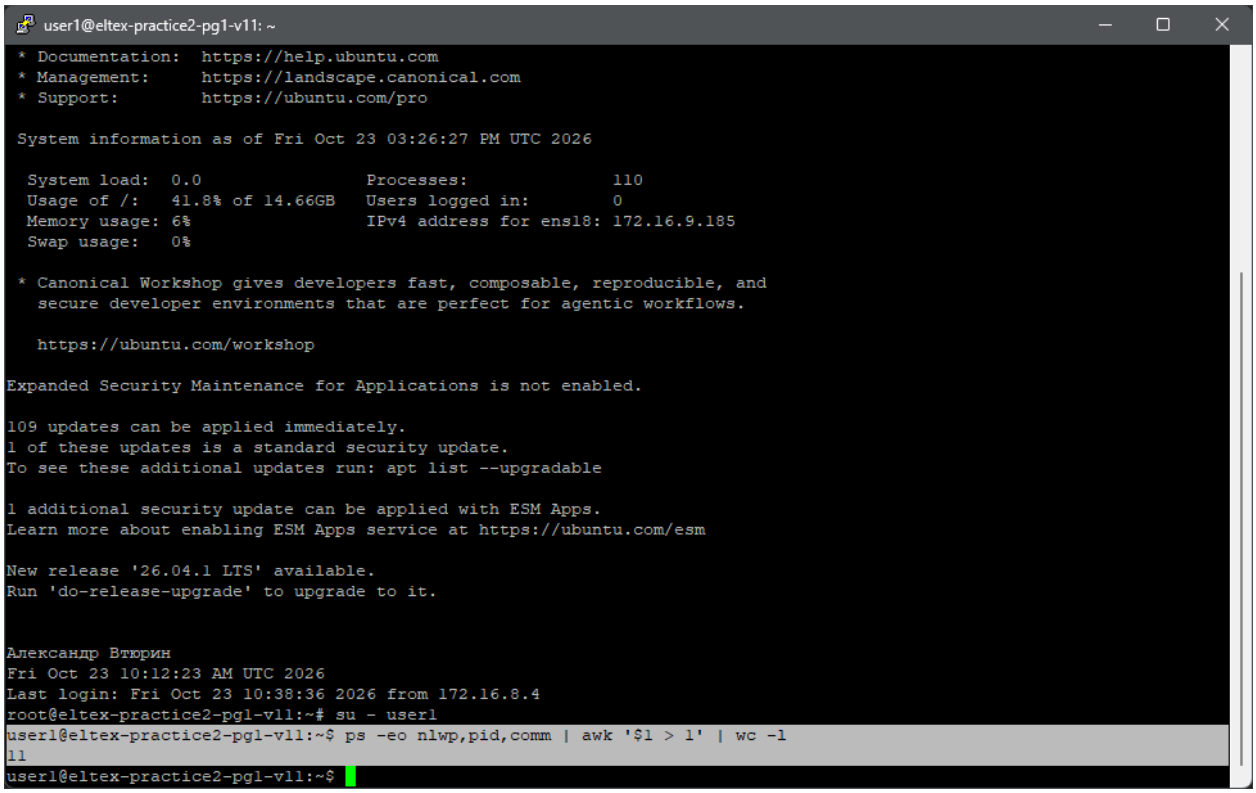
4) В другом терминальном окне выполните команду passwd и оставьте ее в состоянии запроса текущего пароля
5) Перейдите в терминальное окно с top и выполните следующие действия:
   • выведите все процессы, для которых реальным пользователем является пользователь, которым вы вошли в сеанс;
   • найдите процесс, запущенный командой passwd;
   • отправьте этому процессу сигналы SIGTERM, SIGINT, SIGQUIT, SIGKILL используя их числовые идентификаторы (man 7 signal)
6) Выполните команду vim ~/file_task3.txt и нажмите Ctrl-Z
7) Выполните команду sleep 600, нажмите Ctrl-Z и выполните команду jobs
8) Последнее задание (sleep 600) сделайте фоновым
9) Измените число NICE у задания (sleep 600), сделав его равным 10
10) Проверьте, что число NICE у этого задания изменилось
11) Сделайте задание vim ~/file_ task3.txt активным и выйдите из редактора
12) Отправьте сигнал SIGTERM заданию sleep 600 и выполните команду jobs
13) Создайте перехватчик сигналов SIGINT и SIGQUIT внутри командного интерпретатора, который выводит сообщение «Меня голыми руками не возьмёшь!» (используйте встроенную команду trap) и отправьте сигналы самому себе
14) Создайте несколько зомби-процессов несколько раз выполнив команду:
    (sleep 1 & exec /bin/sleep 30) &
    30 – настраиваемая задержка времени существования зомби-процесса в секундах, если не успеваете выполнить следующий пункт – увеличьте время
15) Найдите при помощи команды ps все зомби-процессы, выведите их PID и PPID. Попробуйте досрочно завершить выполнение зомби-процессов.
    Все, что узнали о зомби-процессах и их особенностях в процессе выполнения пунктов опишите используя ~10 предложений в отчете.

1)



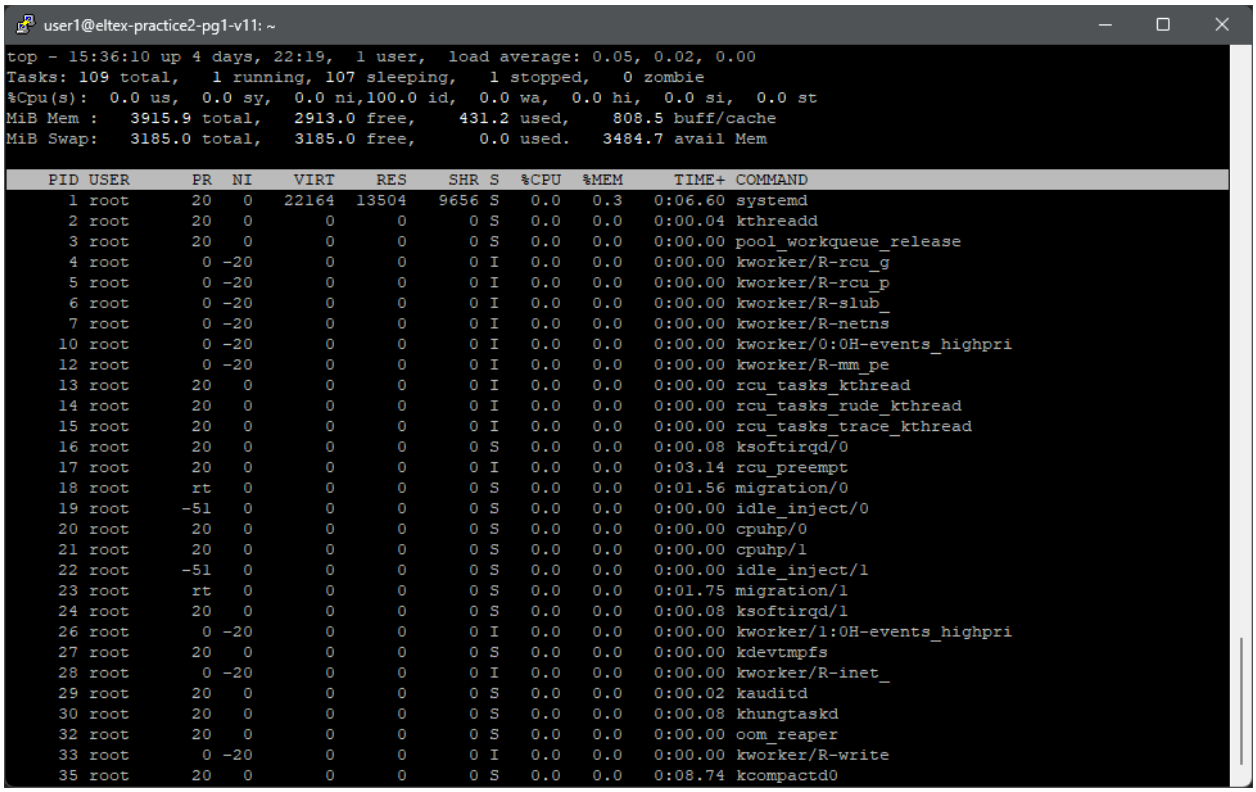
Вход под пользователем user1 из практики 2

2)

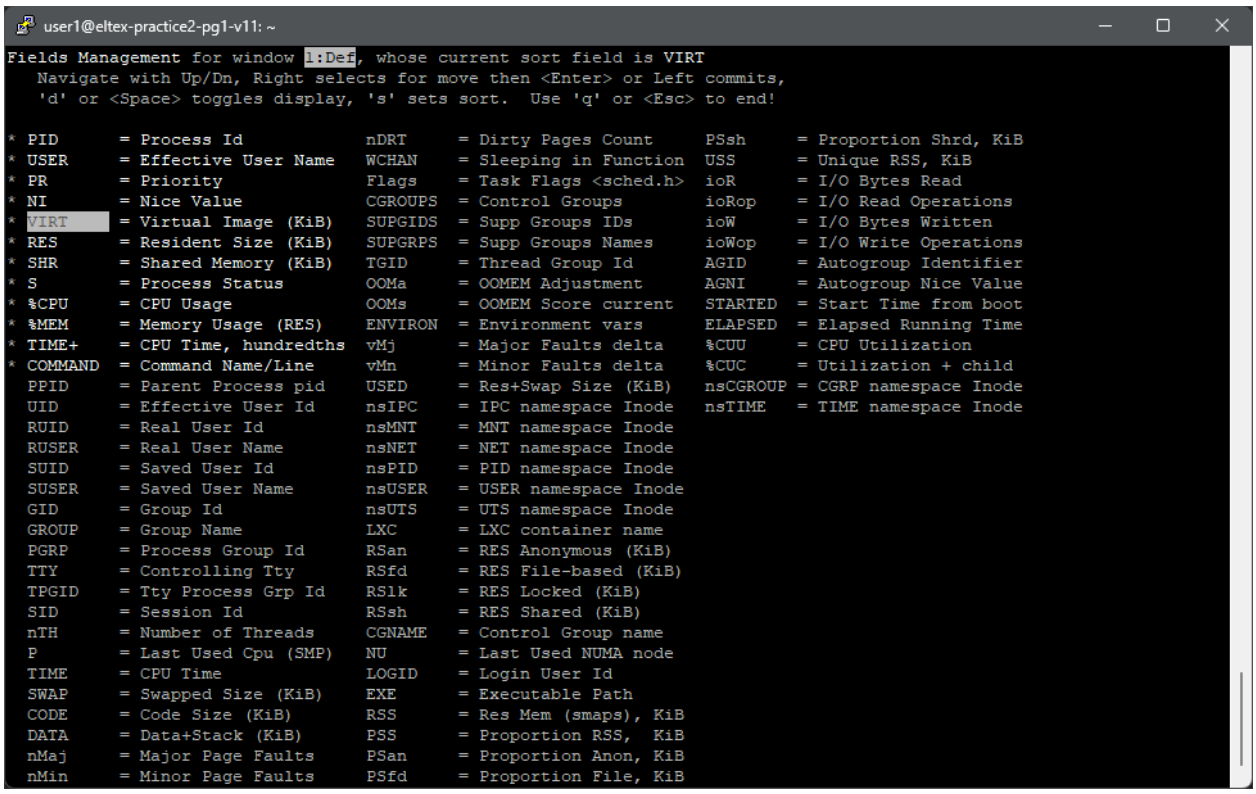


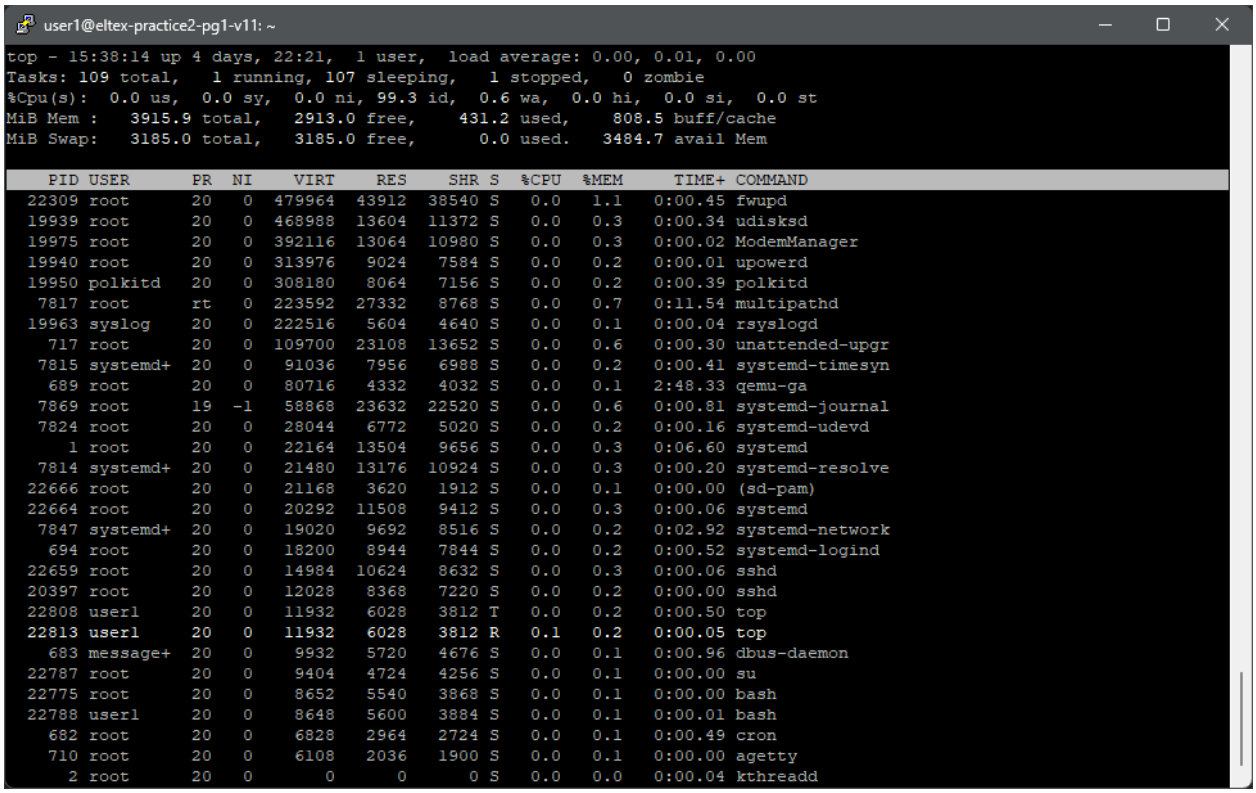
Подсчёт количества процессов, имеющих несколько потоков выполнения

3)

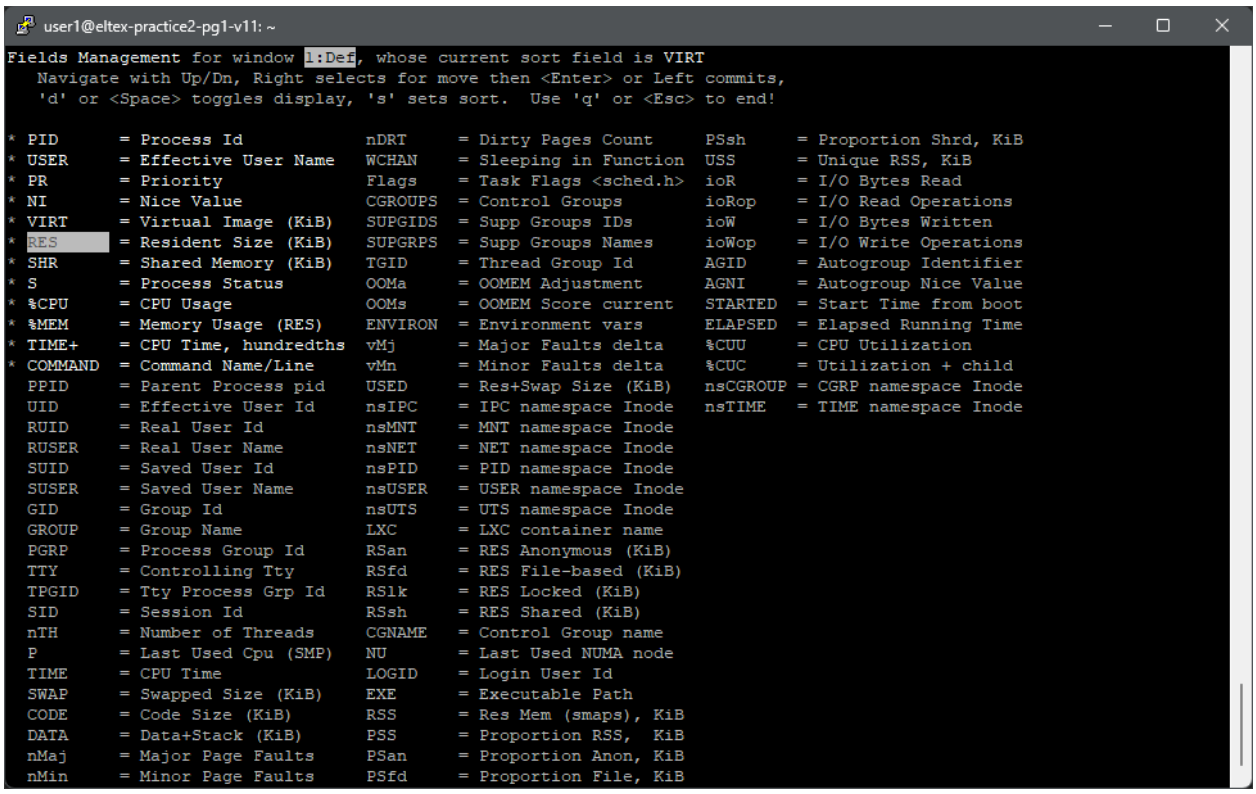


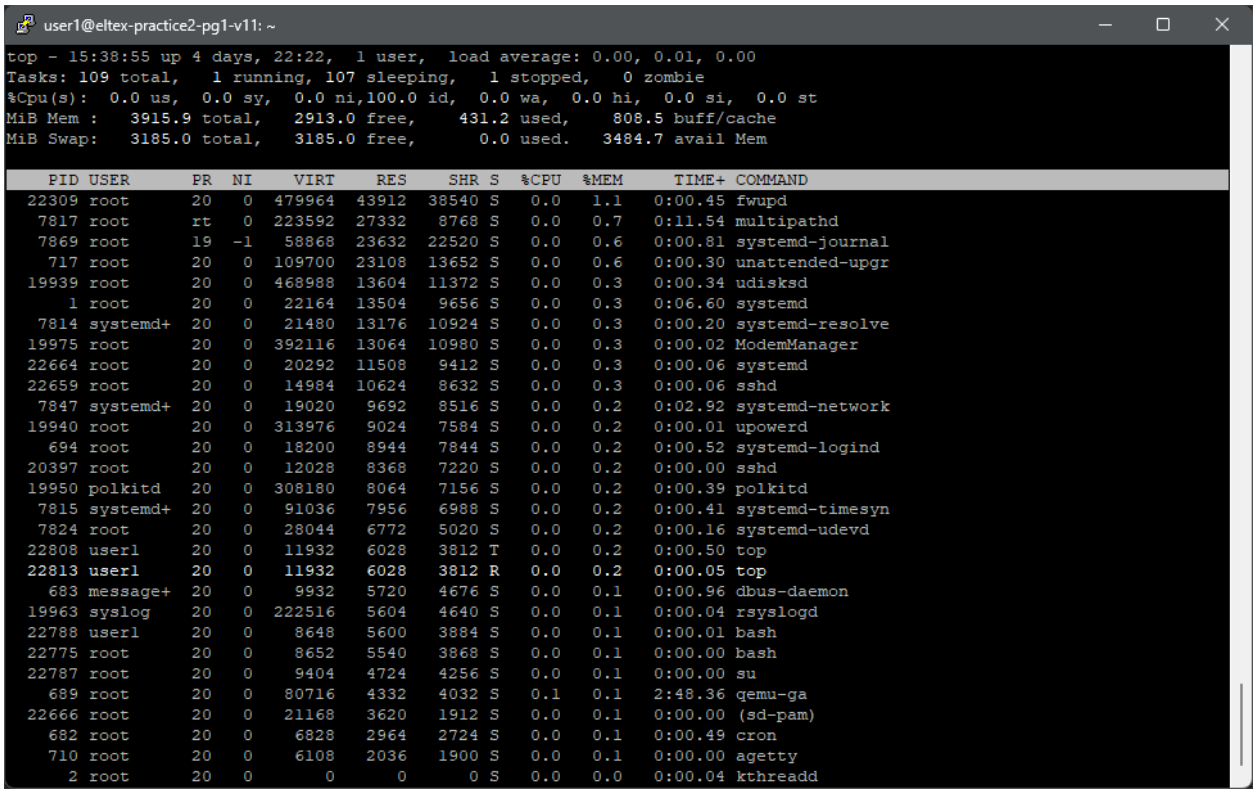
Выполнение команды top



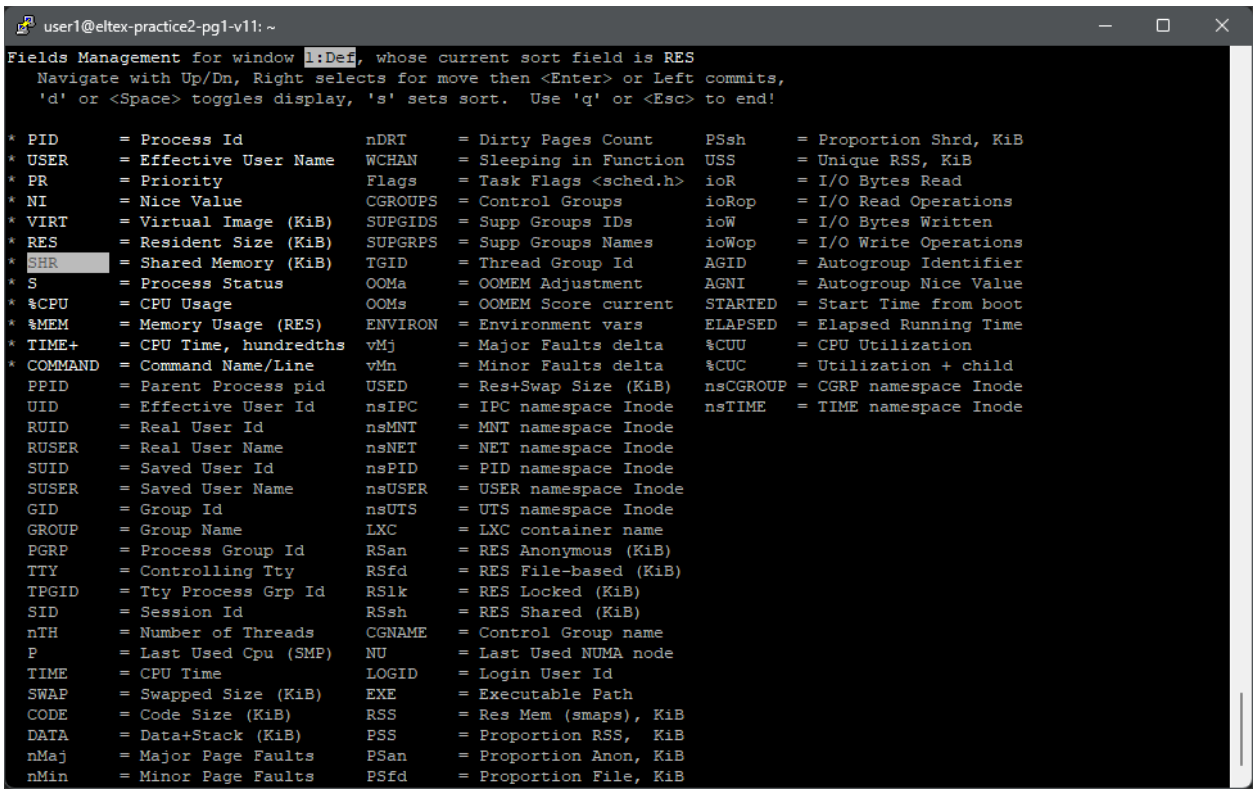


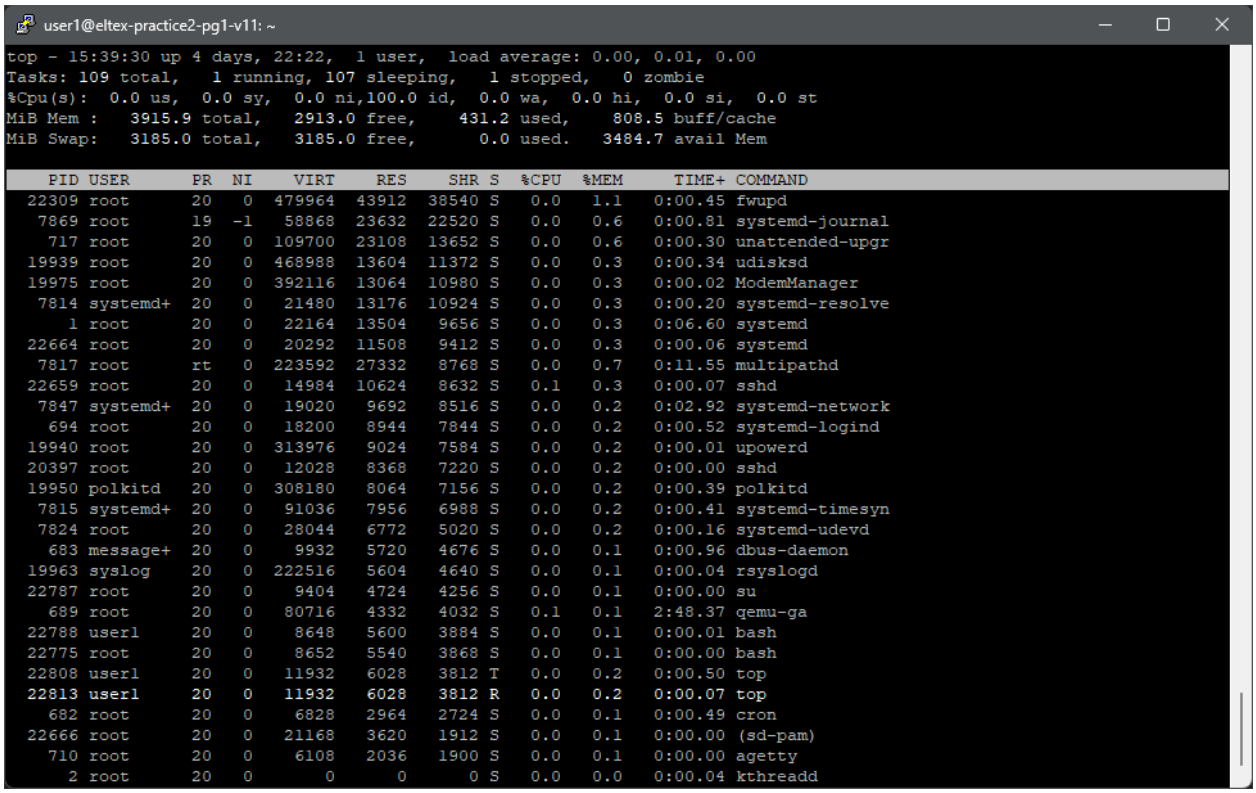
Сортировка значений по убыванию поля VIRT





Сортировка значений по убыванию поля RES

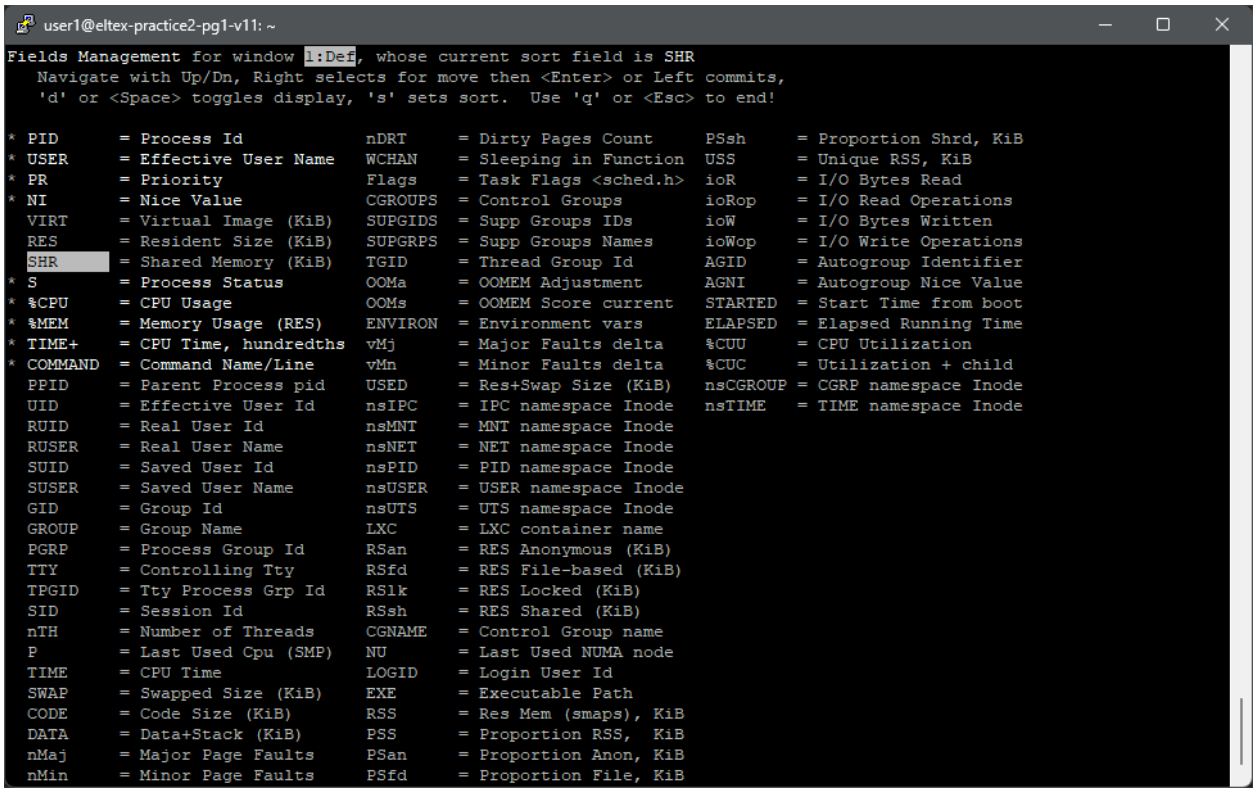




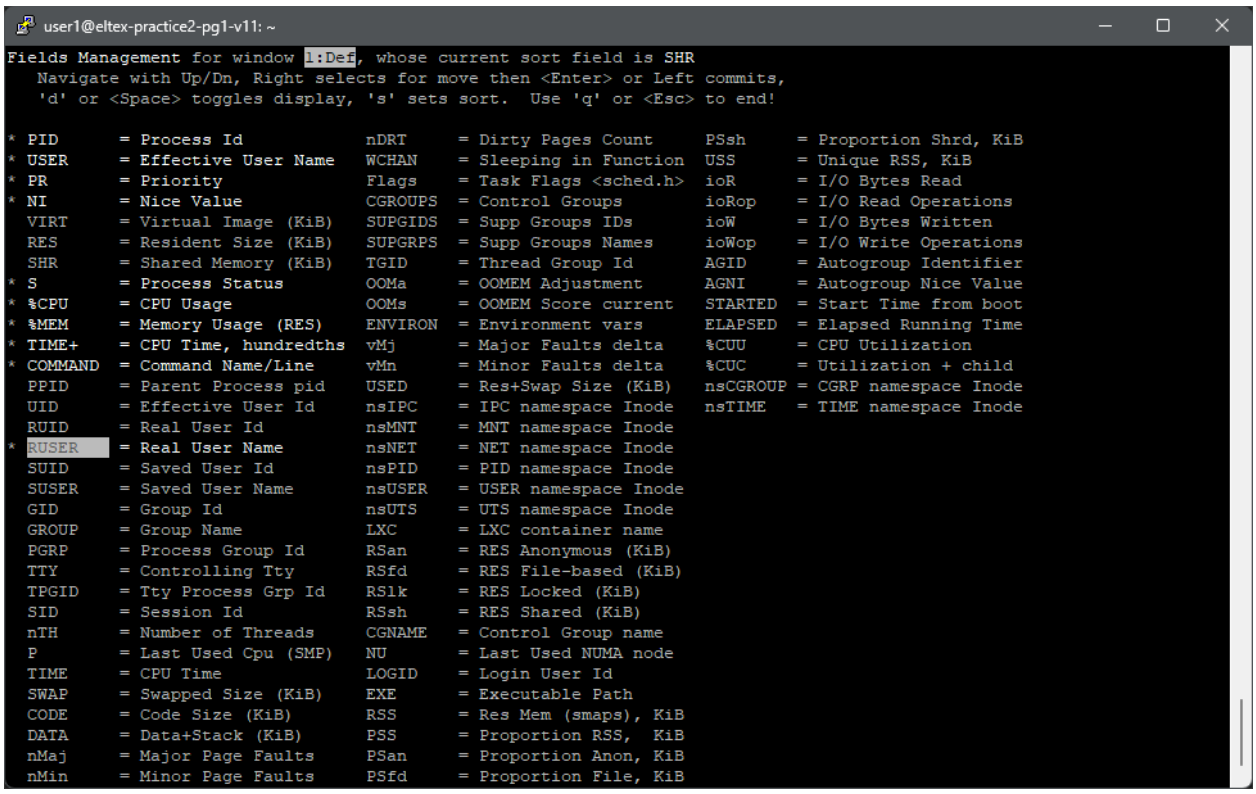
Сортировка значений по убыванию поля SHR

- VIRT (Virtual Memory) — объём виртуального адресного пространства, зарезервированного или видимого для процесса.
- RES (Resident Memory) — резидентная (фактически используемая) память в физической оперативной памяти (RAM). Это объём памяти, действительно загруженный в ОЗУ для данного процесса.

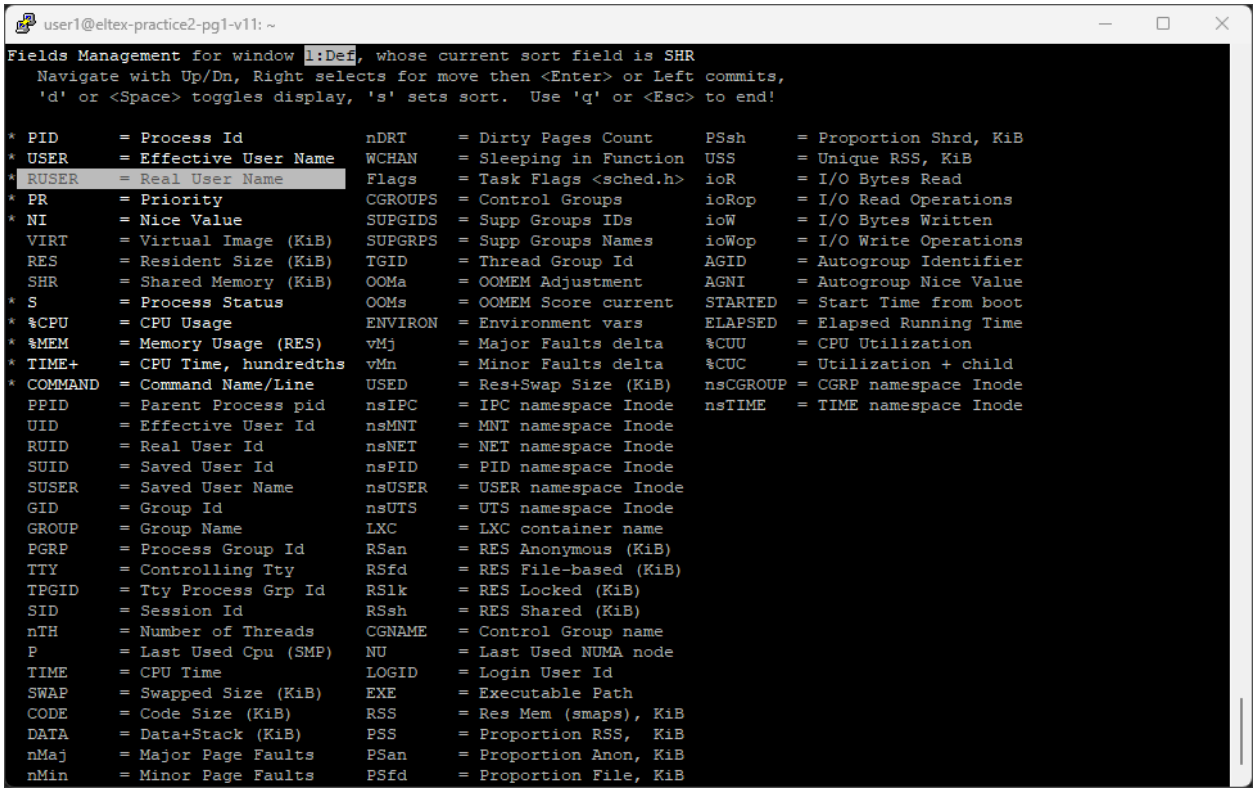
- SHR (Shared Memory) — объём разделяемой памяти, используемой процессом. Обычно это доля памяти, которую процесс использует вместе с другими (например, разделяемые библиотеки, общие сегменты). Если два процесса используют одну и ту же библиотеку, её часть может отображаться в SHR у обоих, но в действительности она хранится в памяти единоразово.



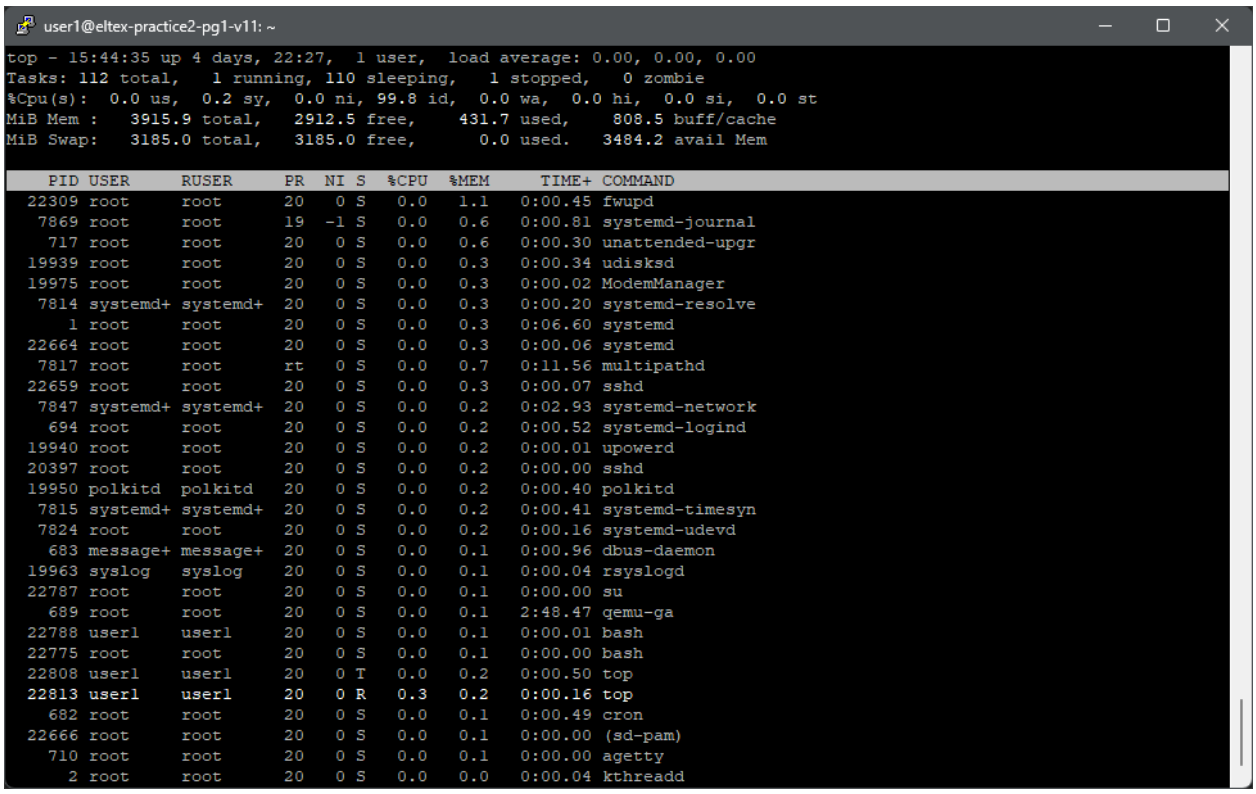
Удаление полей VIRT, RES, SHR



Включение поля RUSER



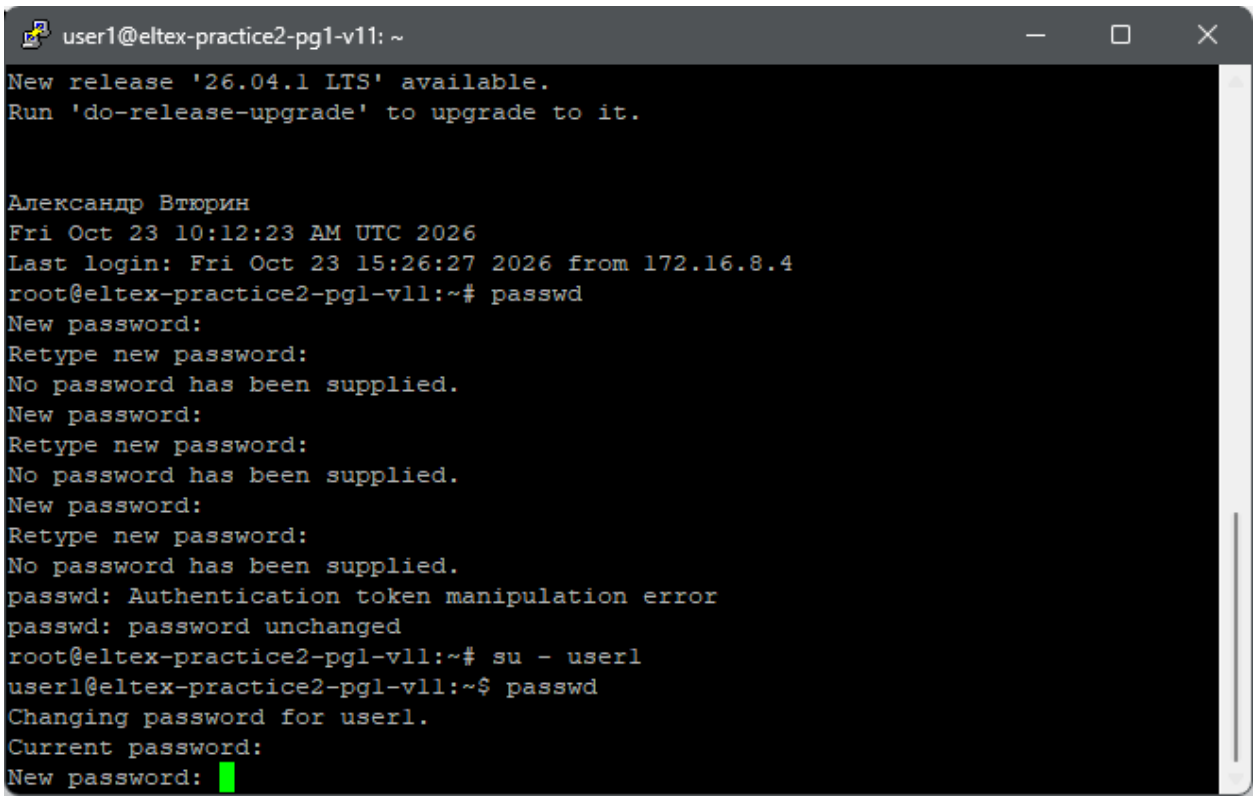
Установка положения поля RUSER



Настроенный вывод команды top

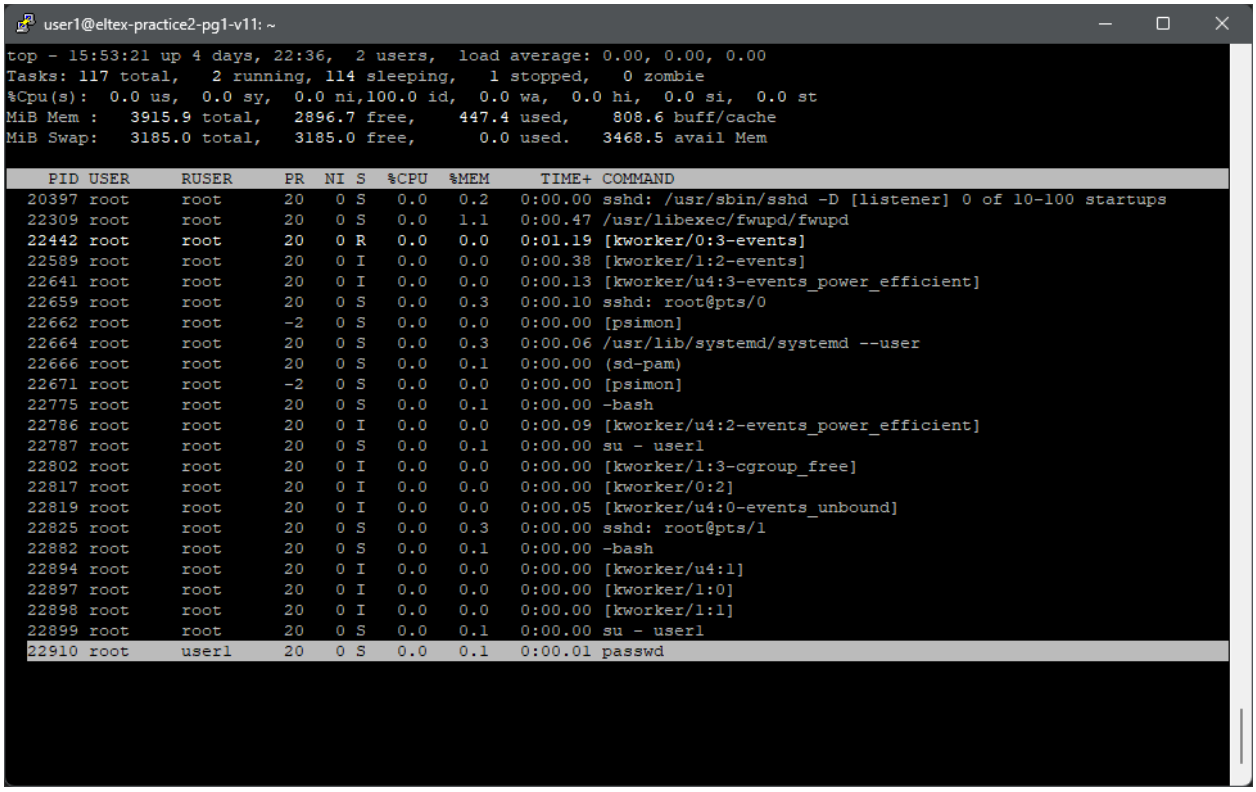
- RUSER (Real User): Имя пользователя, который фактически запустил процесс (владелец учетной записи, от имени которой создана задача).
- USER (Effective User): Эффективное имя пользователя(EUID правами которого процесс наделен в текущий момент). Обычно совпадает с RUSER, но может отличаться, если процесс использует механизм setuid (например, выполняется с временными привилегиями другого пользователя, такого как root).

4)

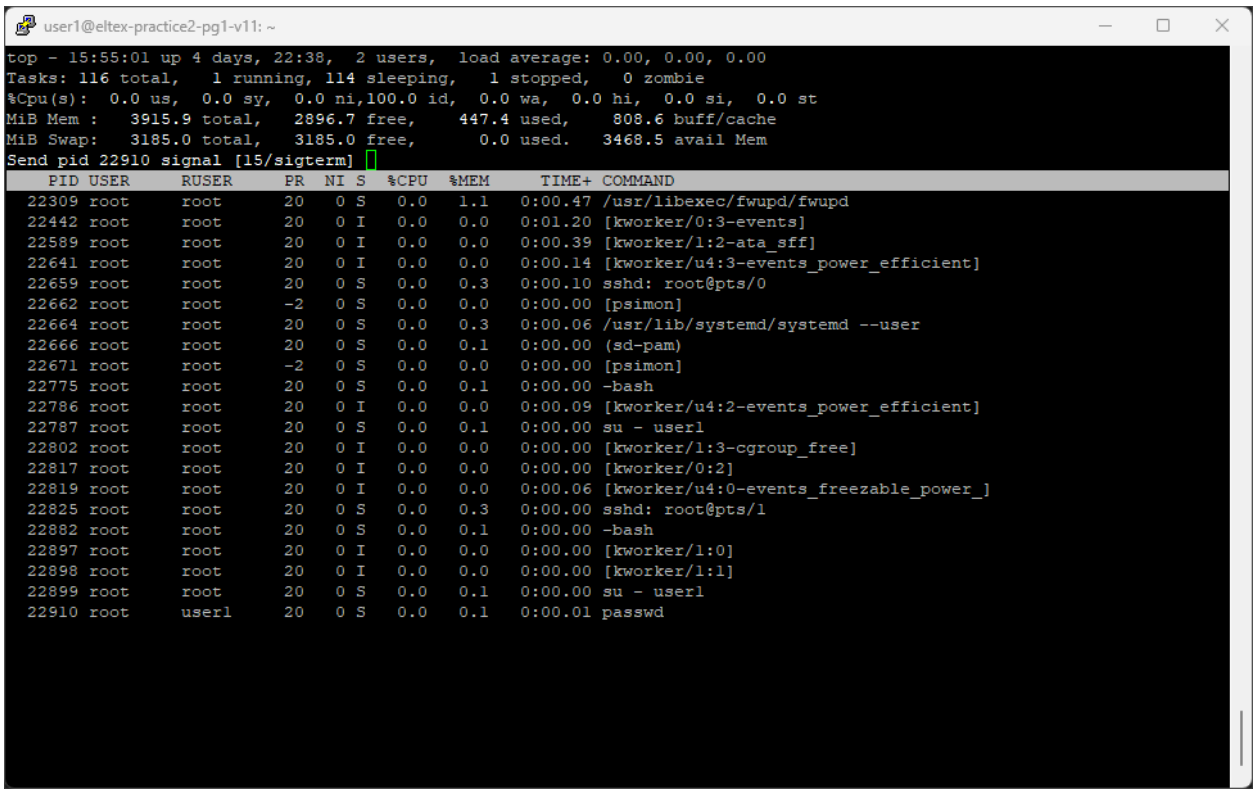


Второе окно с запущенной командой passwd

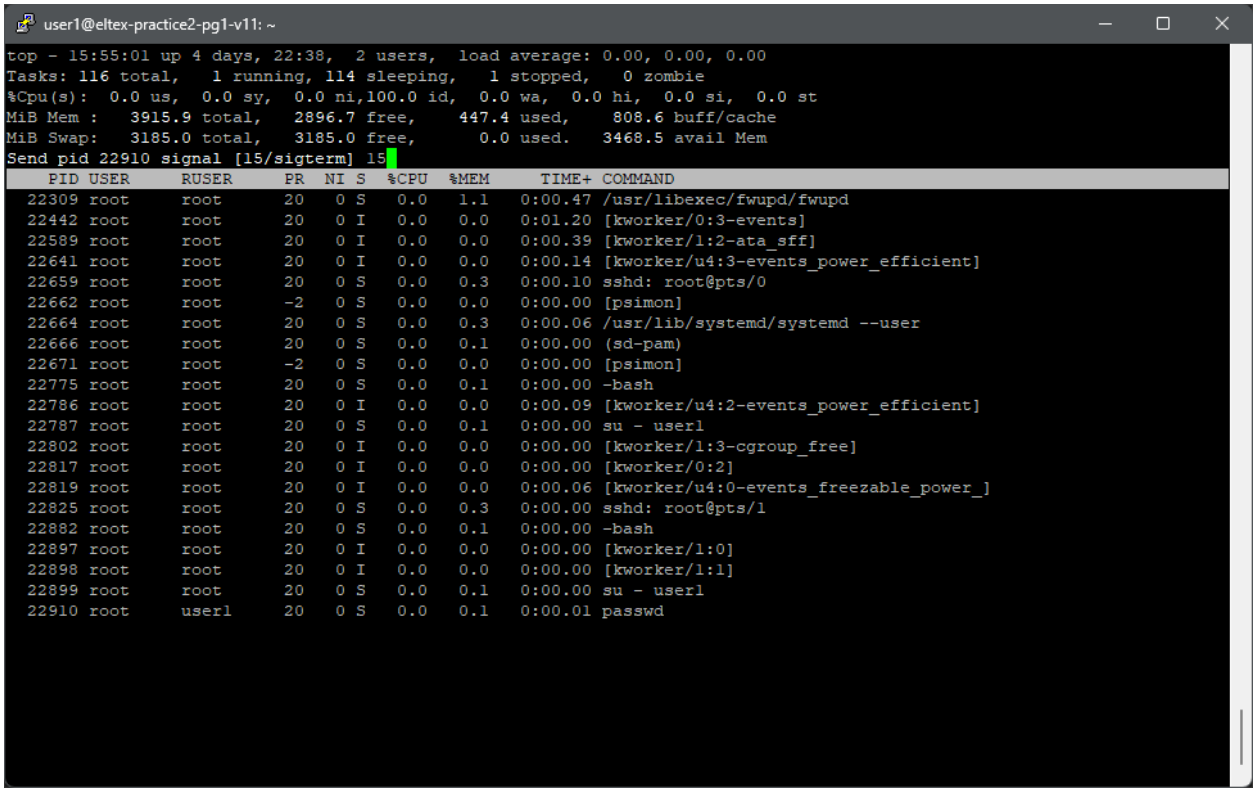
5)



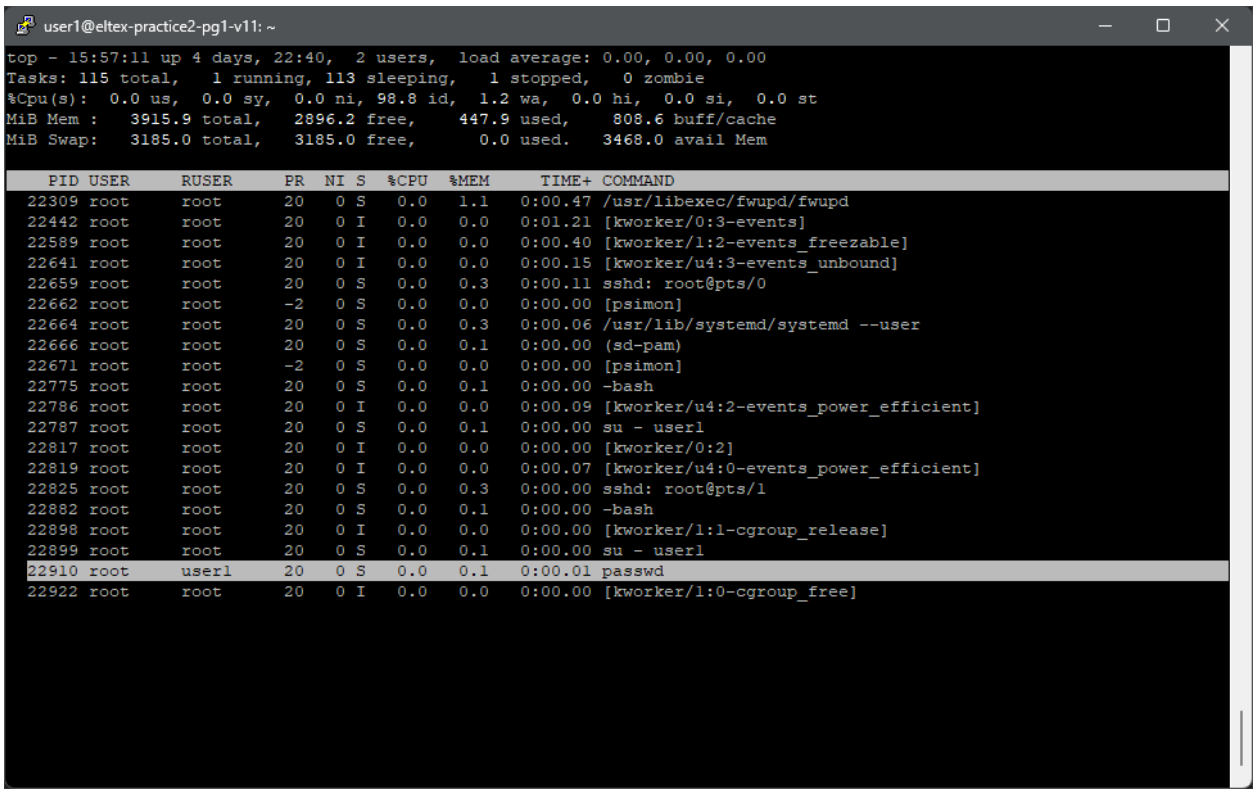
Процесс запущенной команды passwd (PID 22910)



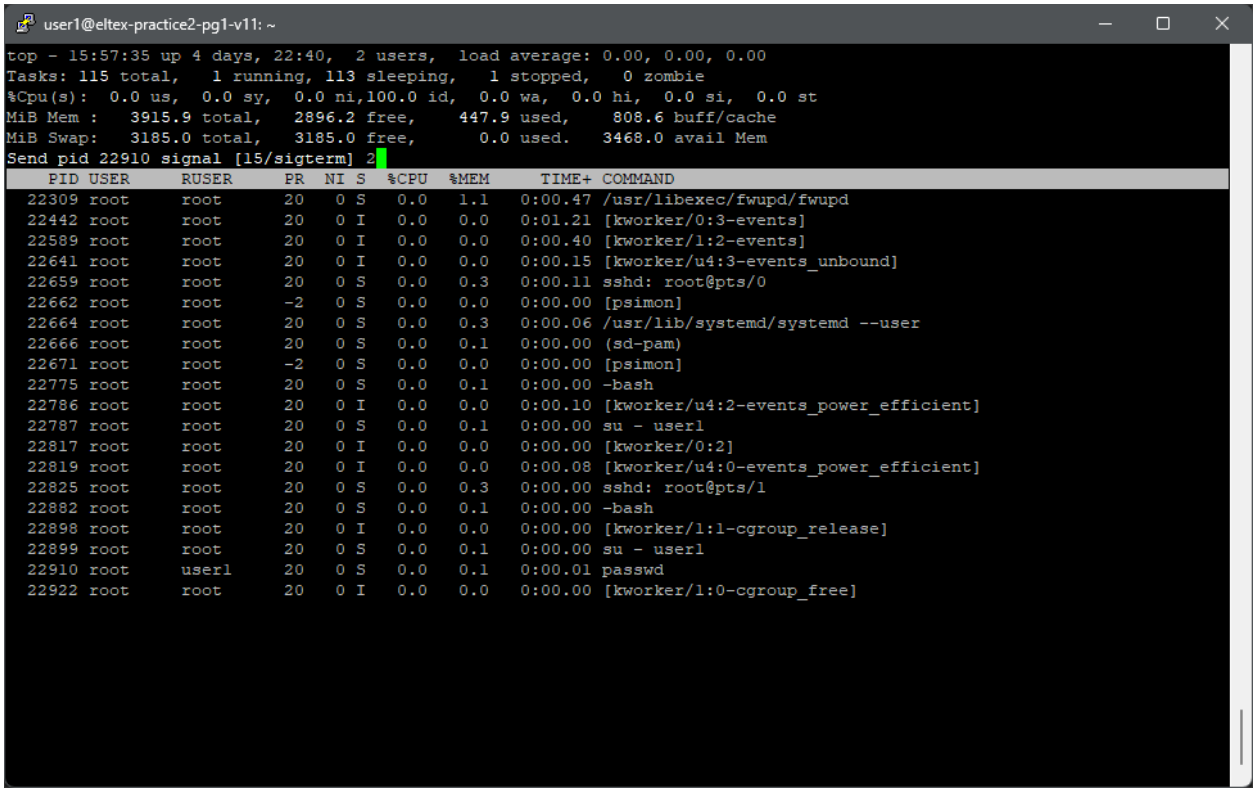
Отправка процессу сигнала



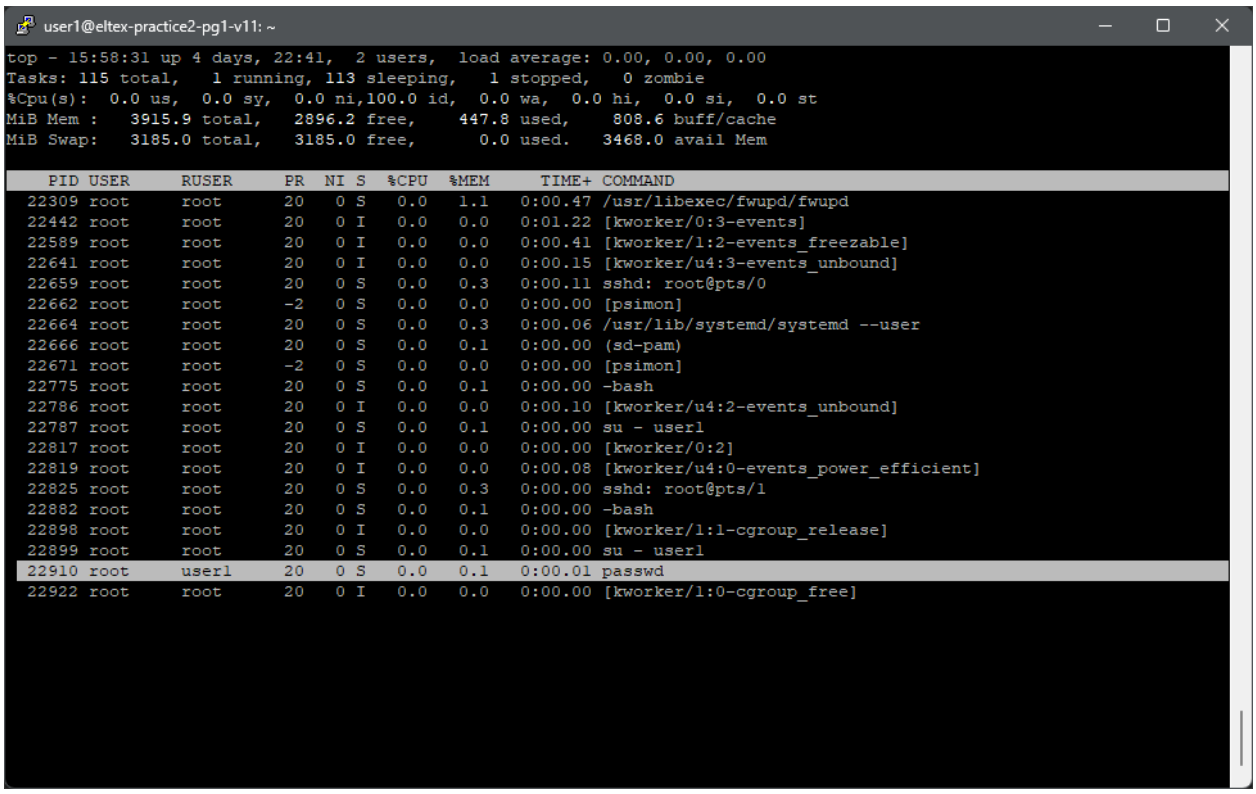
Отправка сигнала SIGTERM



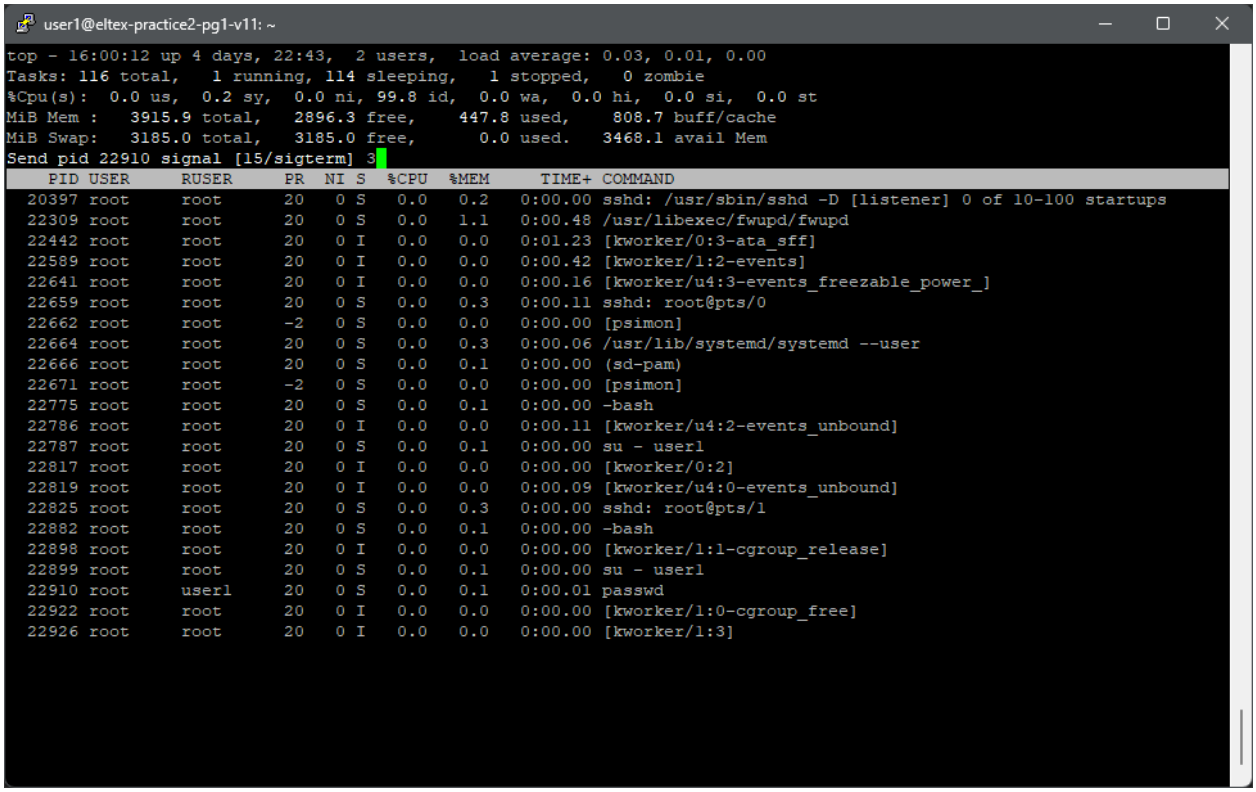
Результат отправки сигнала SIGTERM



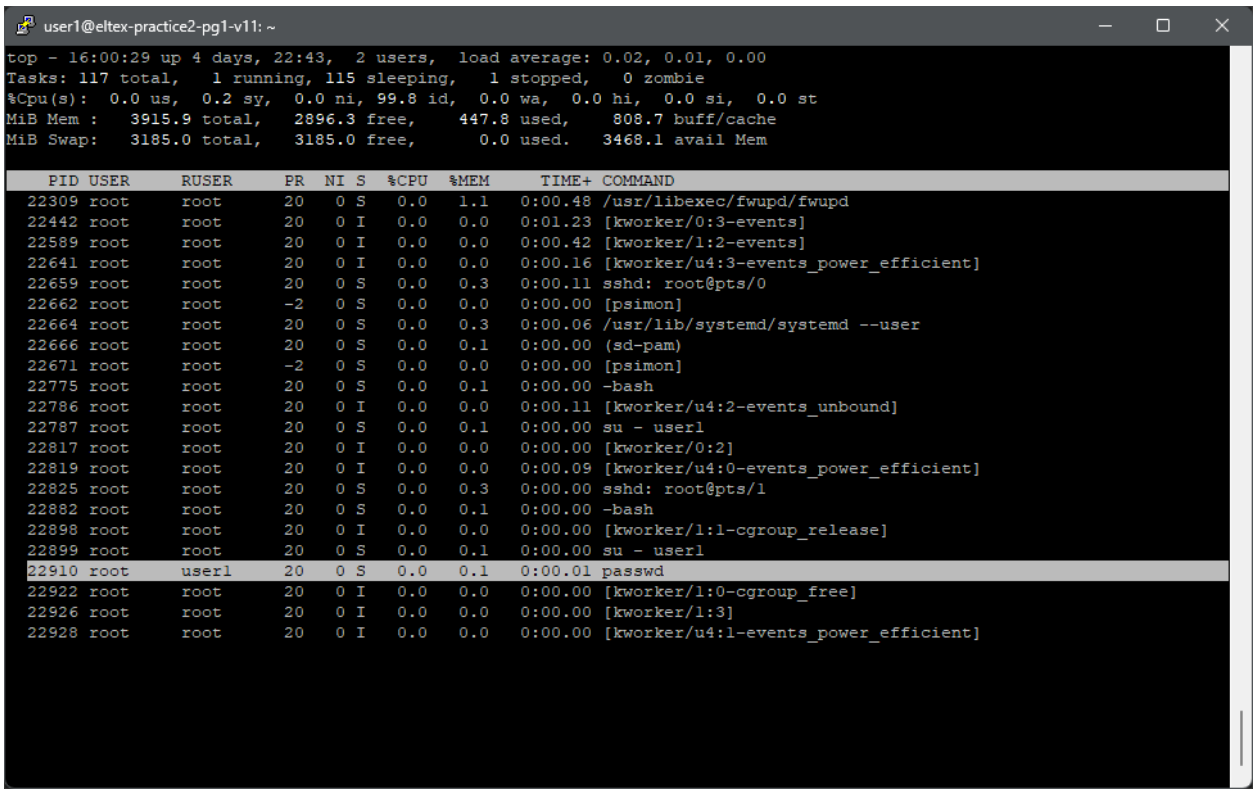
Отправка процессу сигнала SIGINT



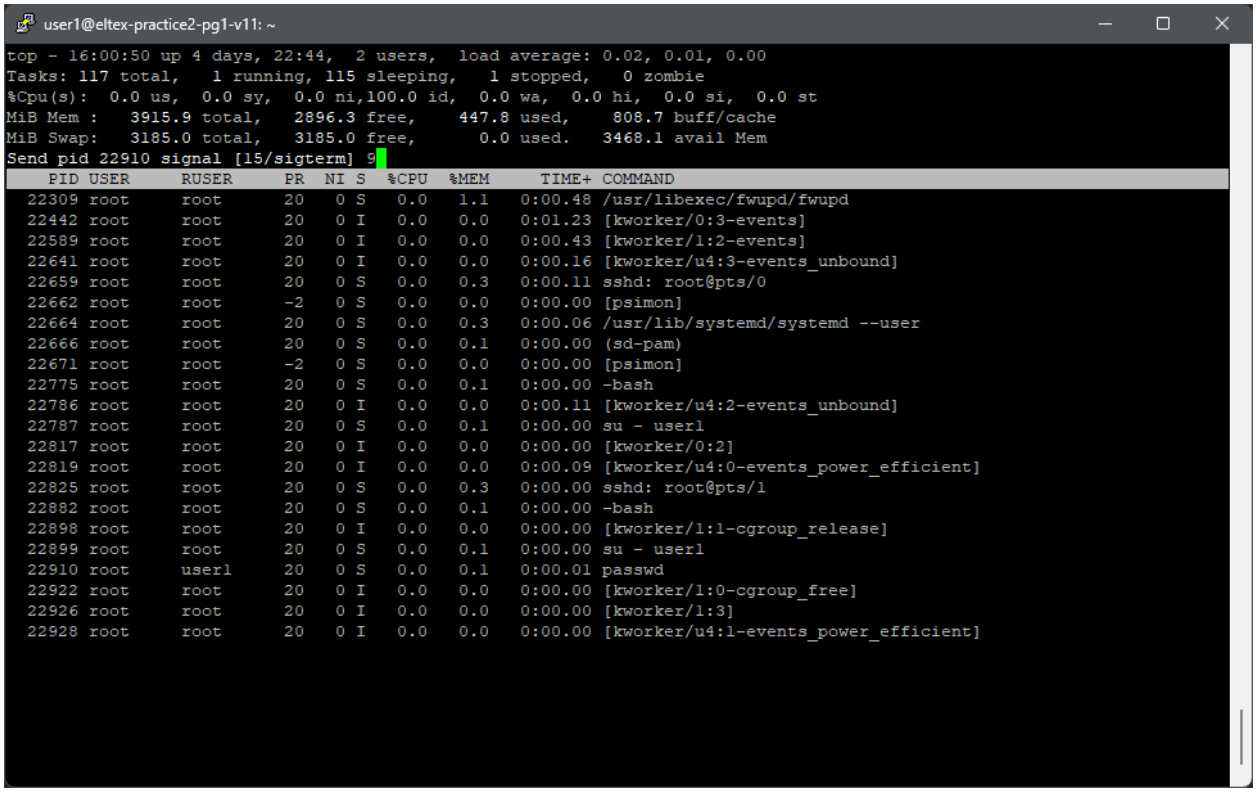
Результат отправки сигнала SIGINT



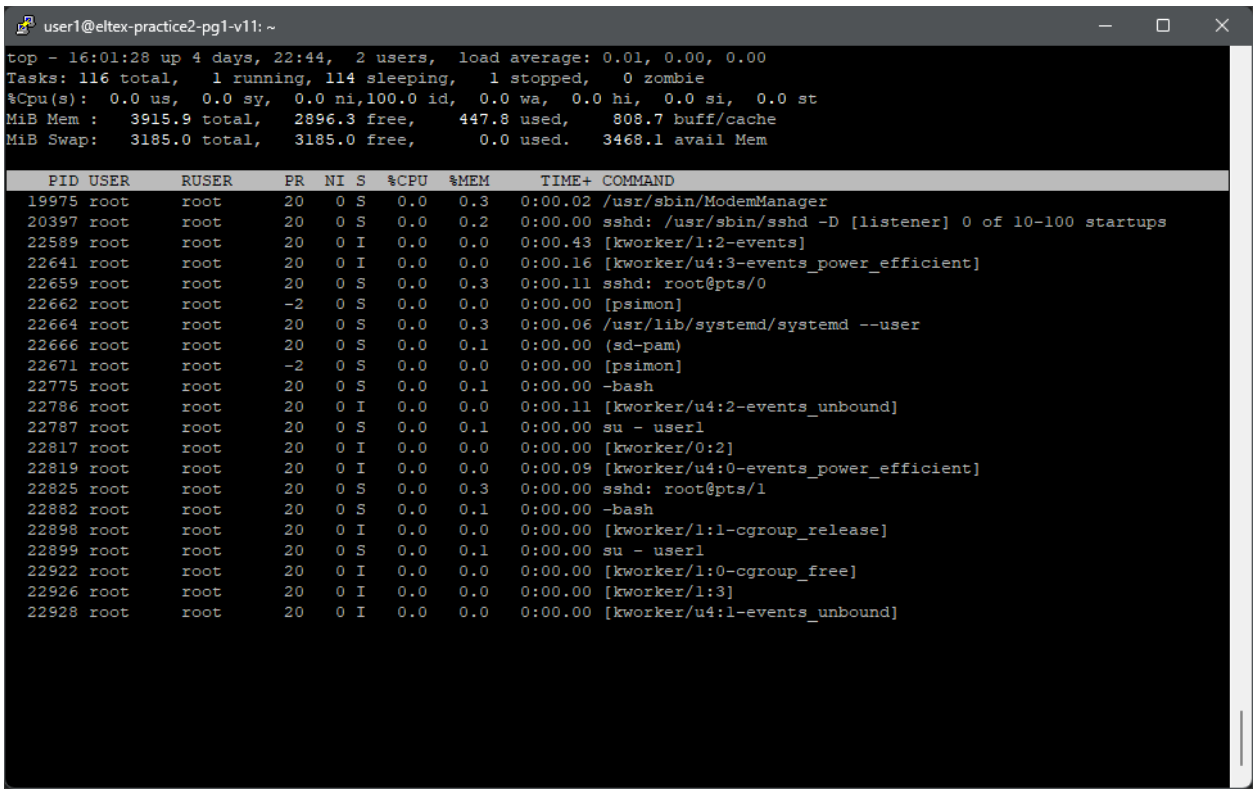
Отправка процессу сигнала SIGQUIT

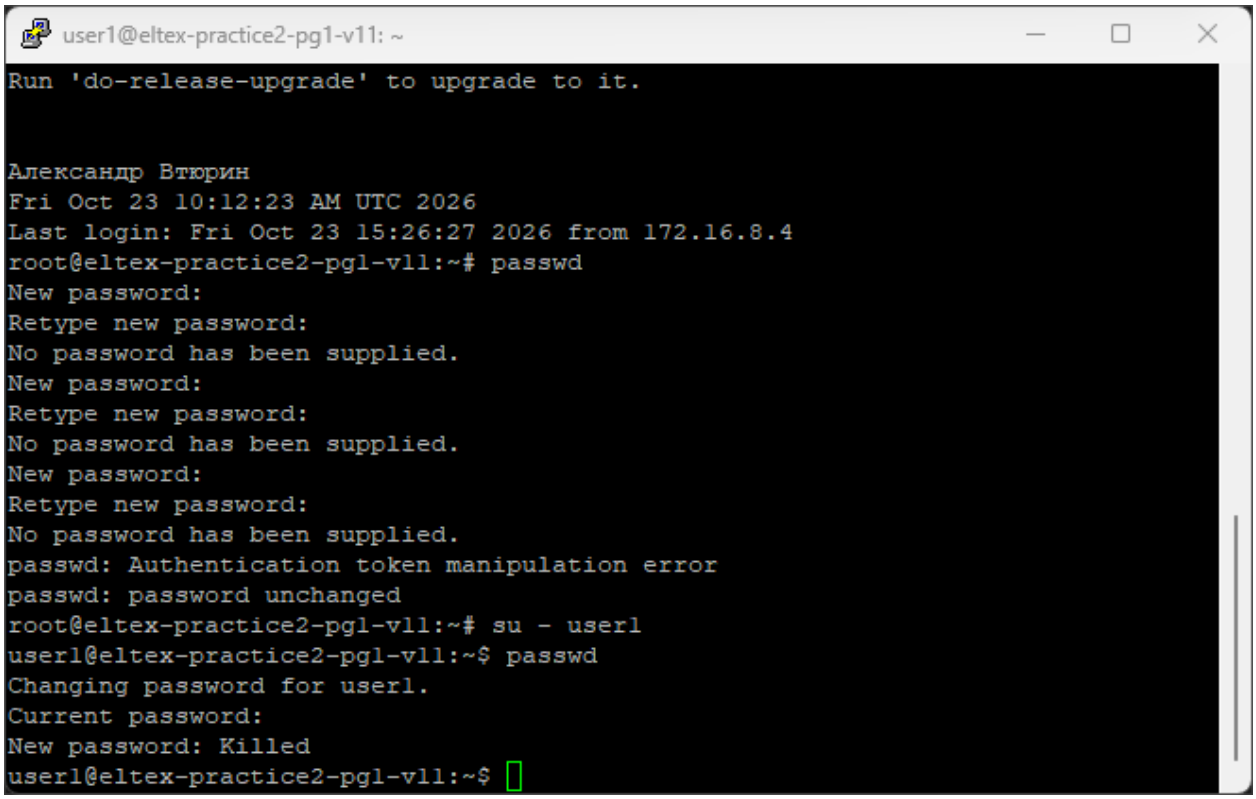


Результат отправки сигнала SIGQUIT



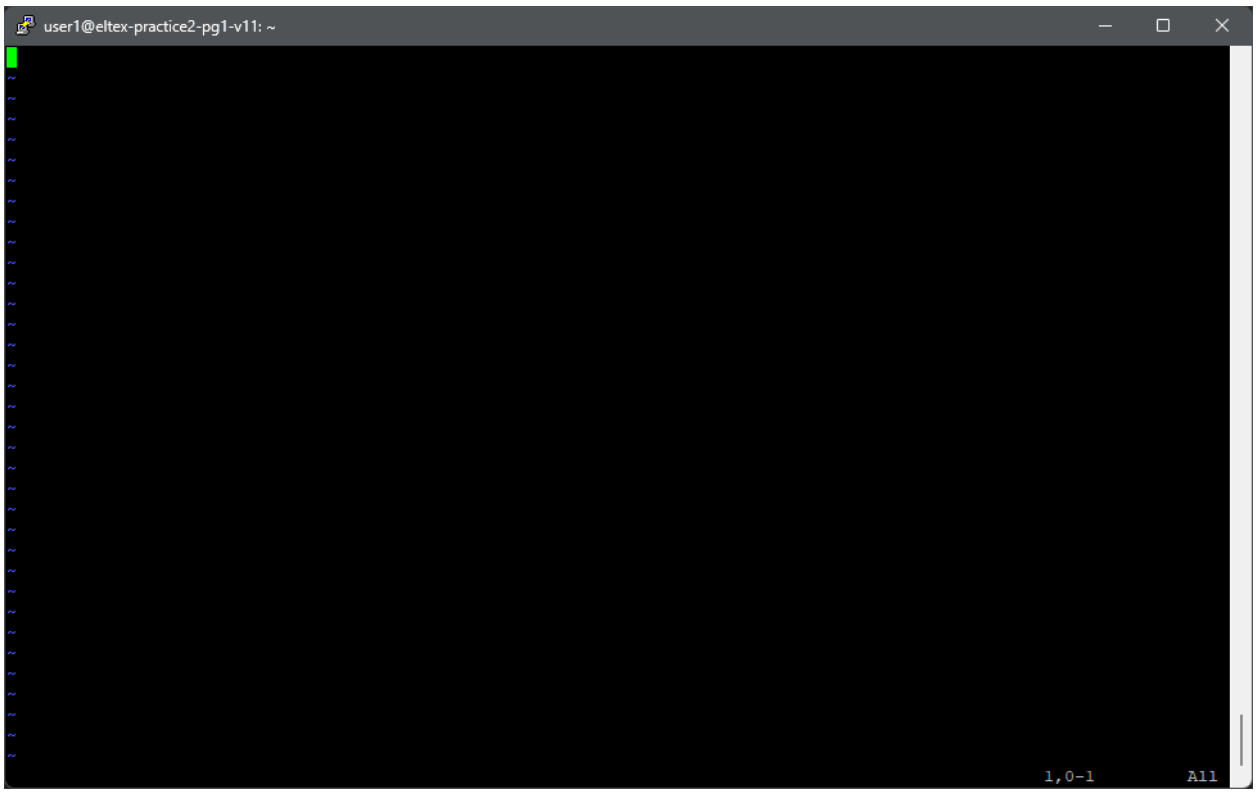
Отправка процессу сигнала SIGKILL



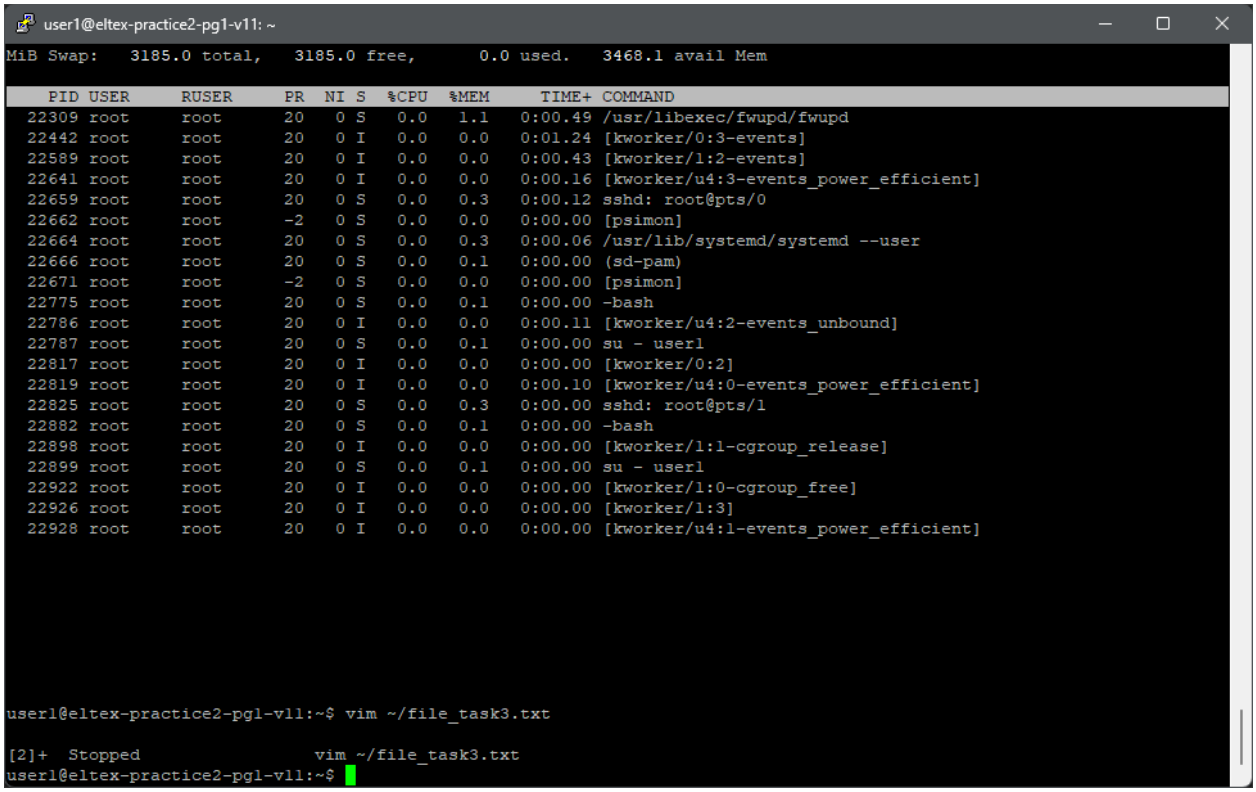


Результат отправки сигнала SIGKILL

6)

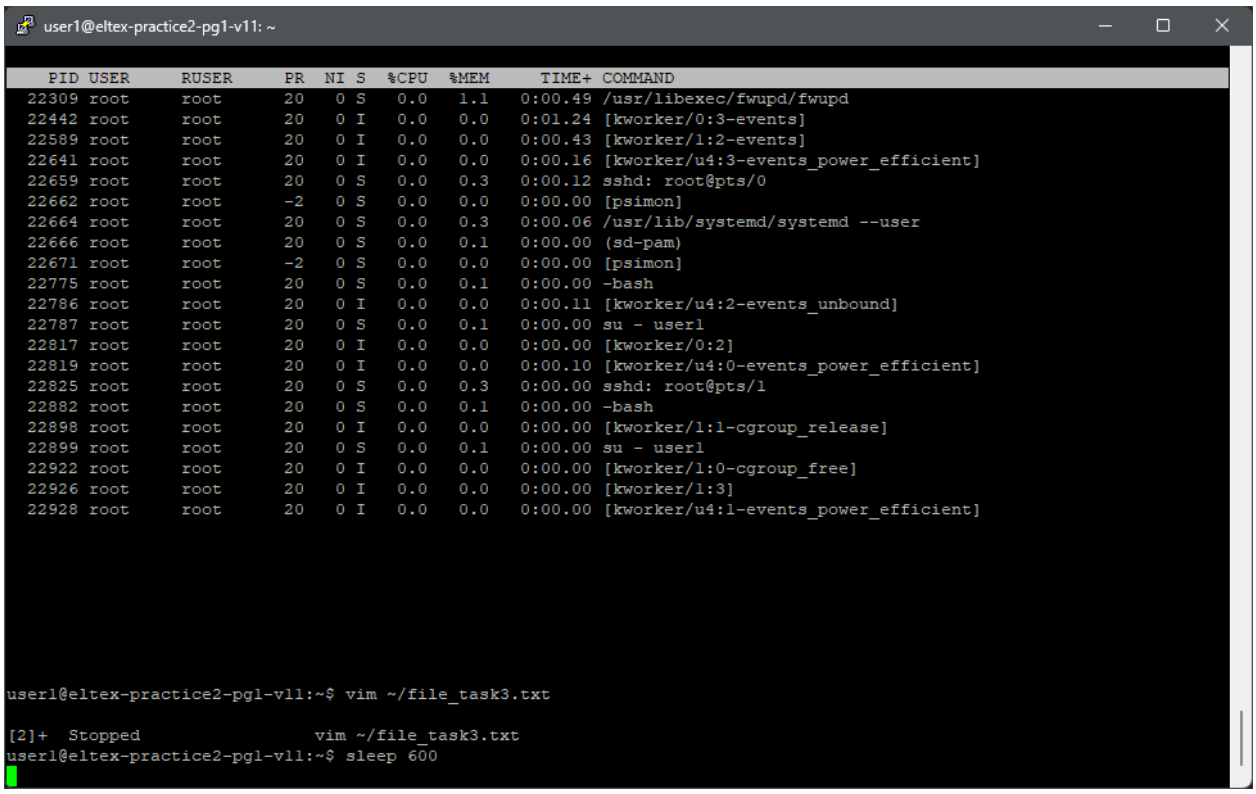


Результат выполнения команды vim ~/file_task3.txt

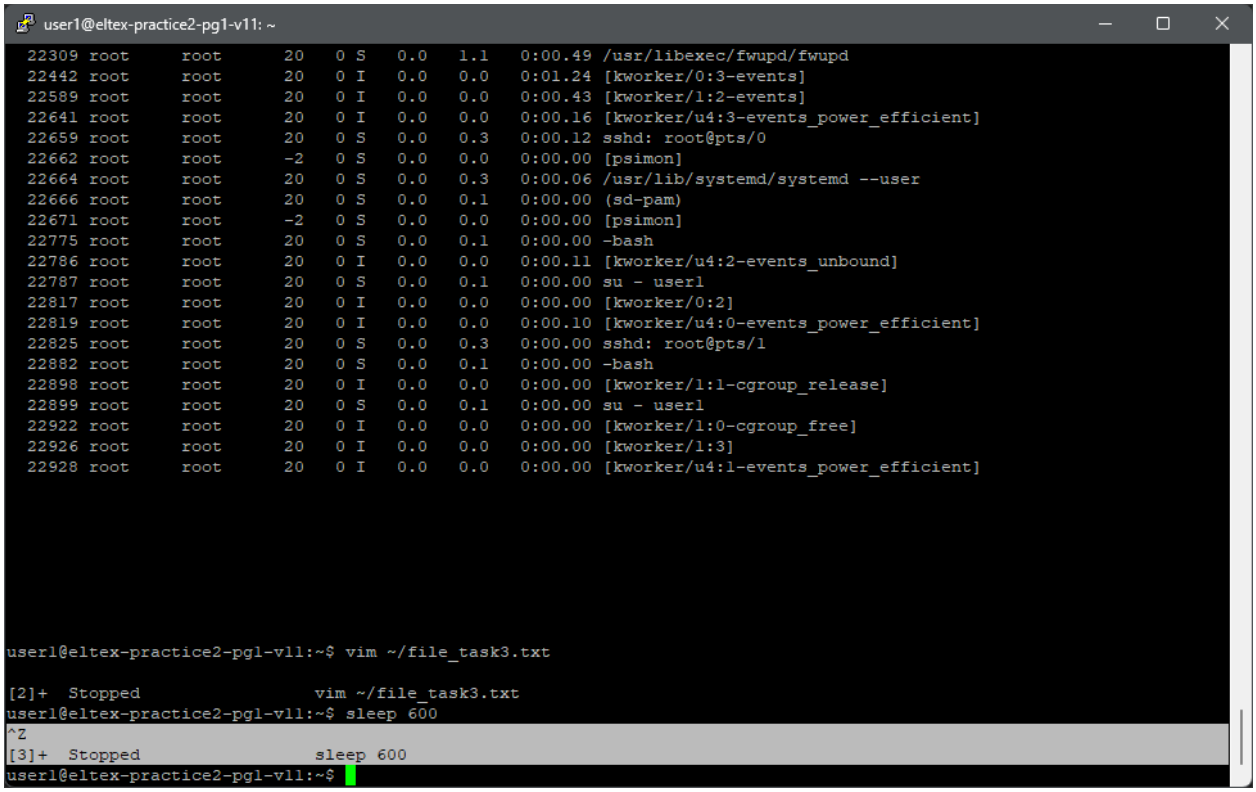


Результат применения Ctrl-Z (процесс прервался)

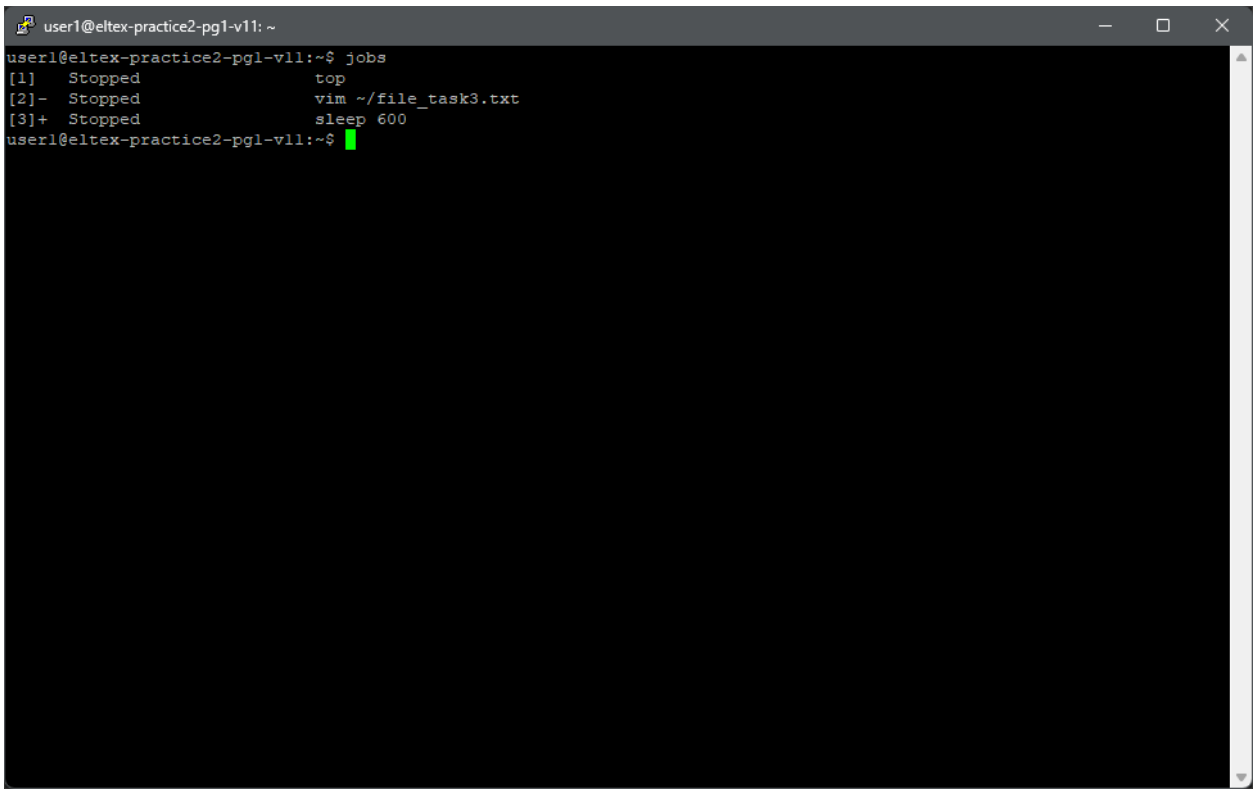
7)



Выполнение команды sleep 600

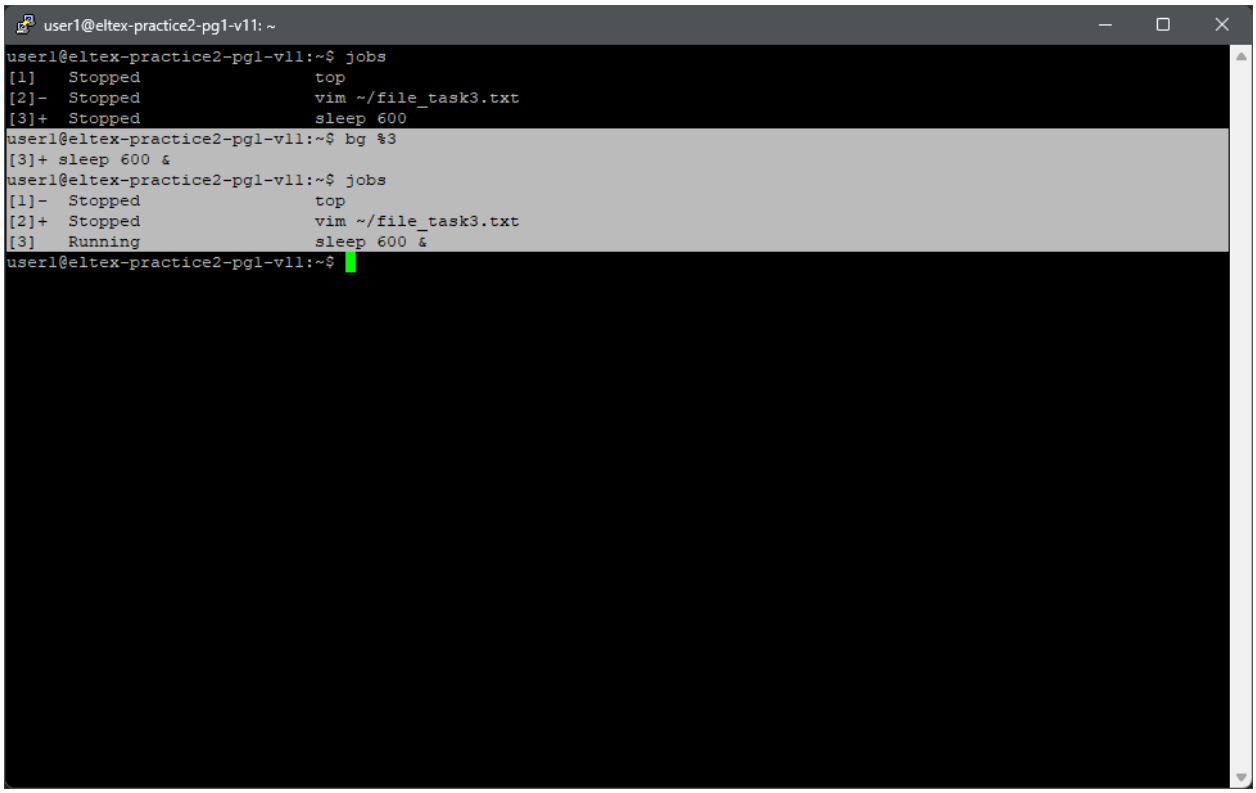


Прерывание выполнения команды через Ctrl-Z



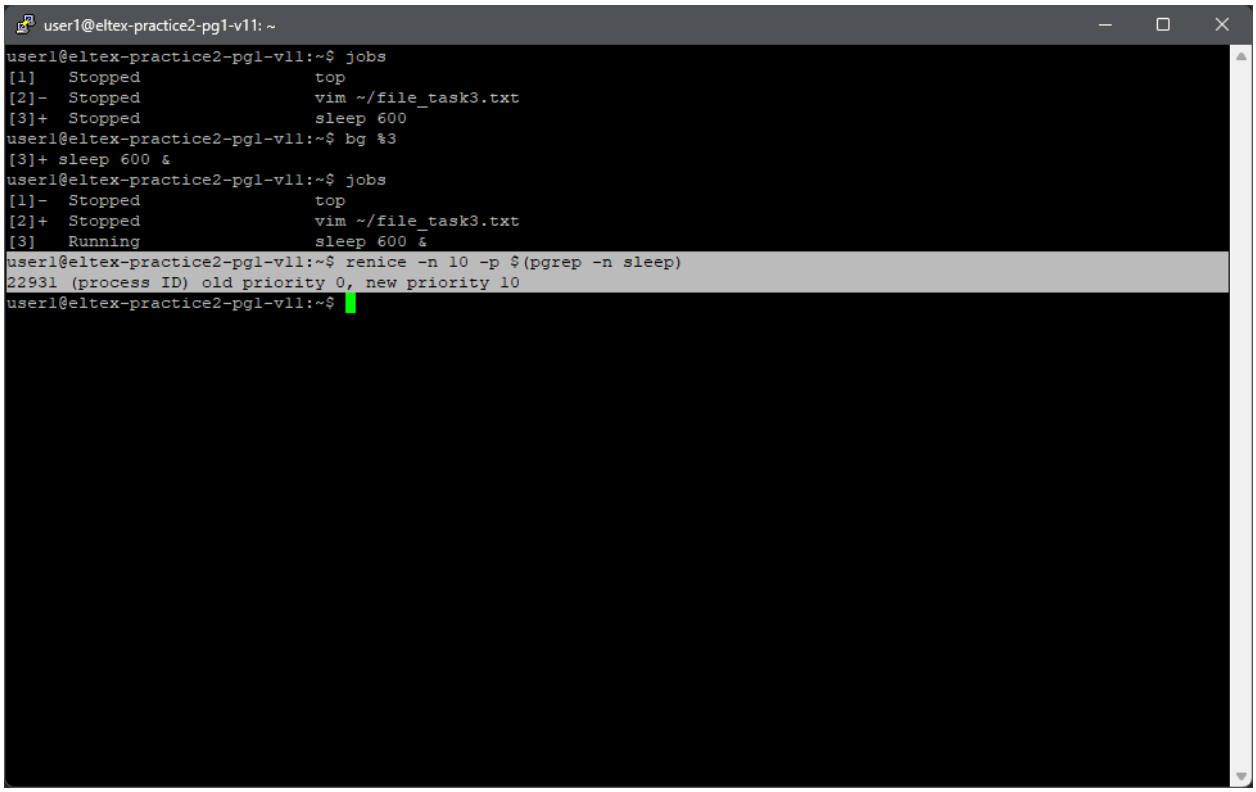
Результат выполнения команды jobs

8)



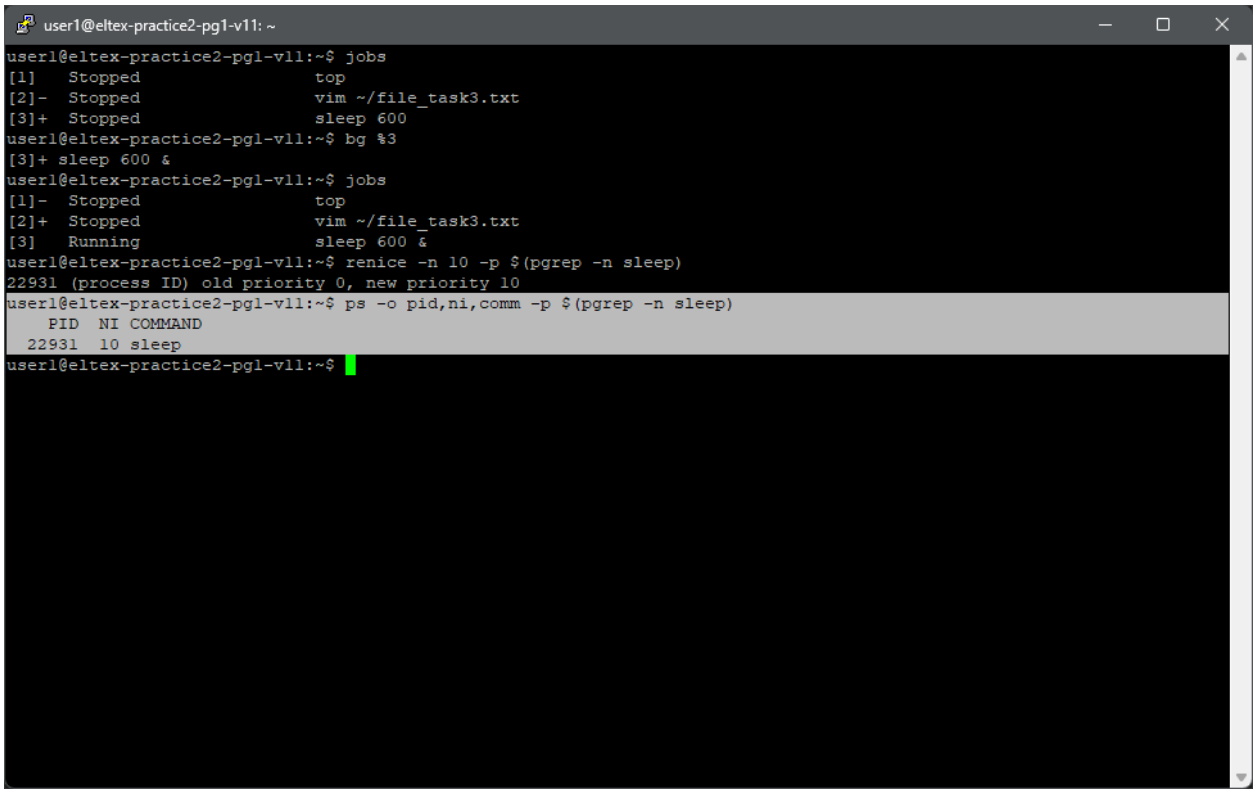
Перевод задания sleep 600 в фоновый режим

9)



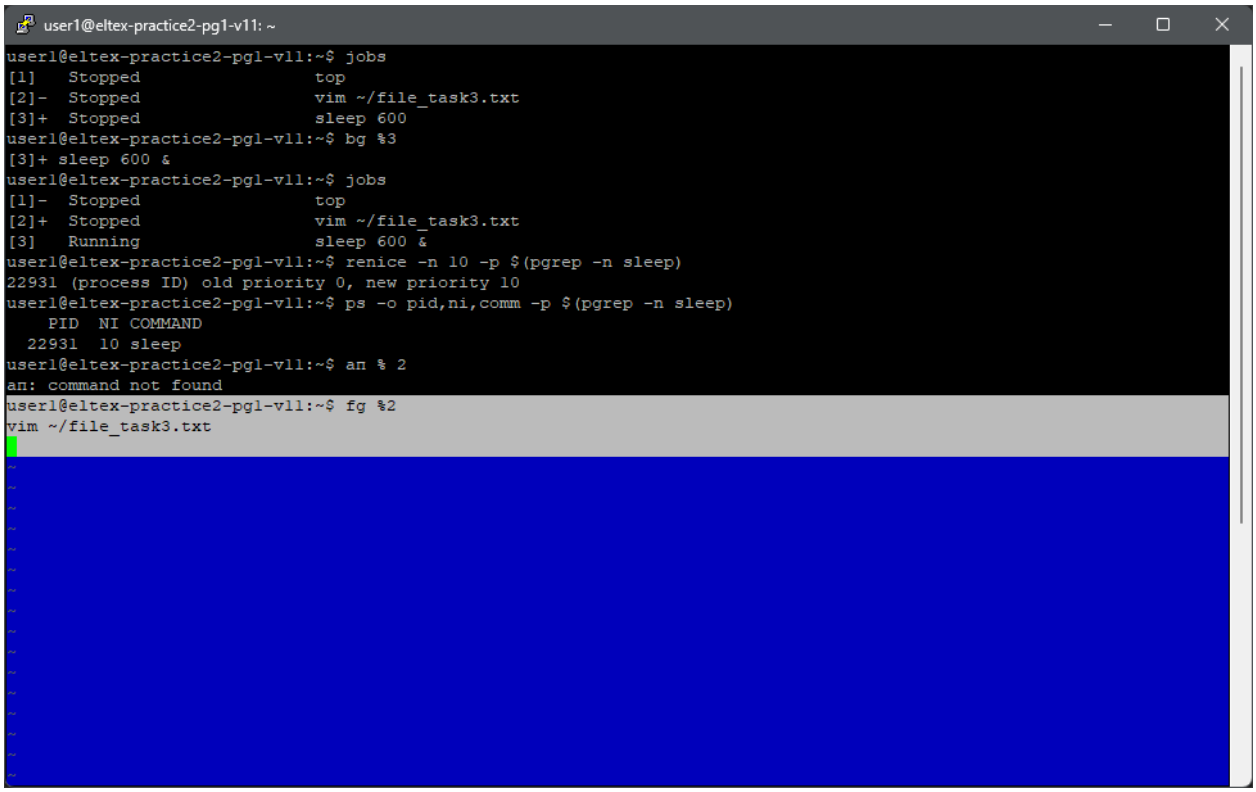
Изменение числа NICE задания sleep 600 на 10

10)

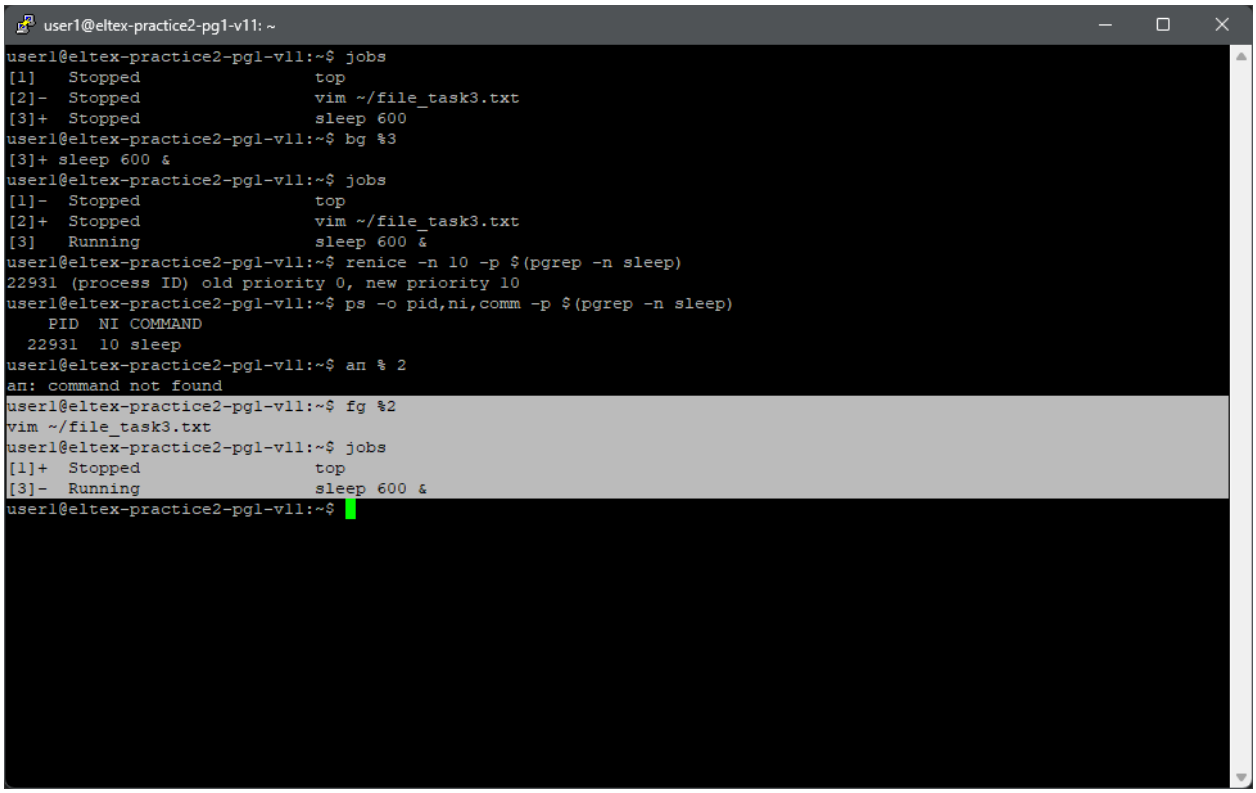


Проверка числа NICE задания sleep 600

11)

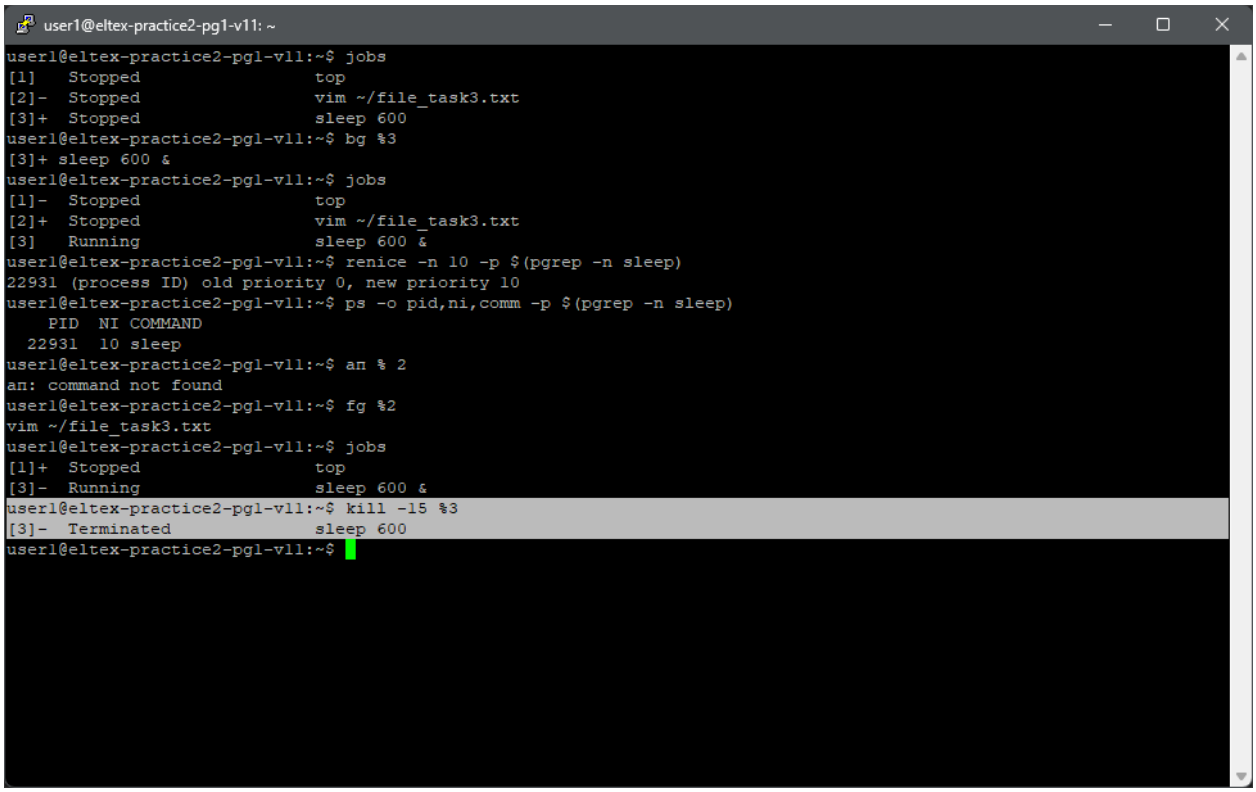


Возвращение процесса vim ~/file_ task3.txt в активный режим

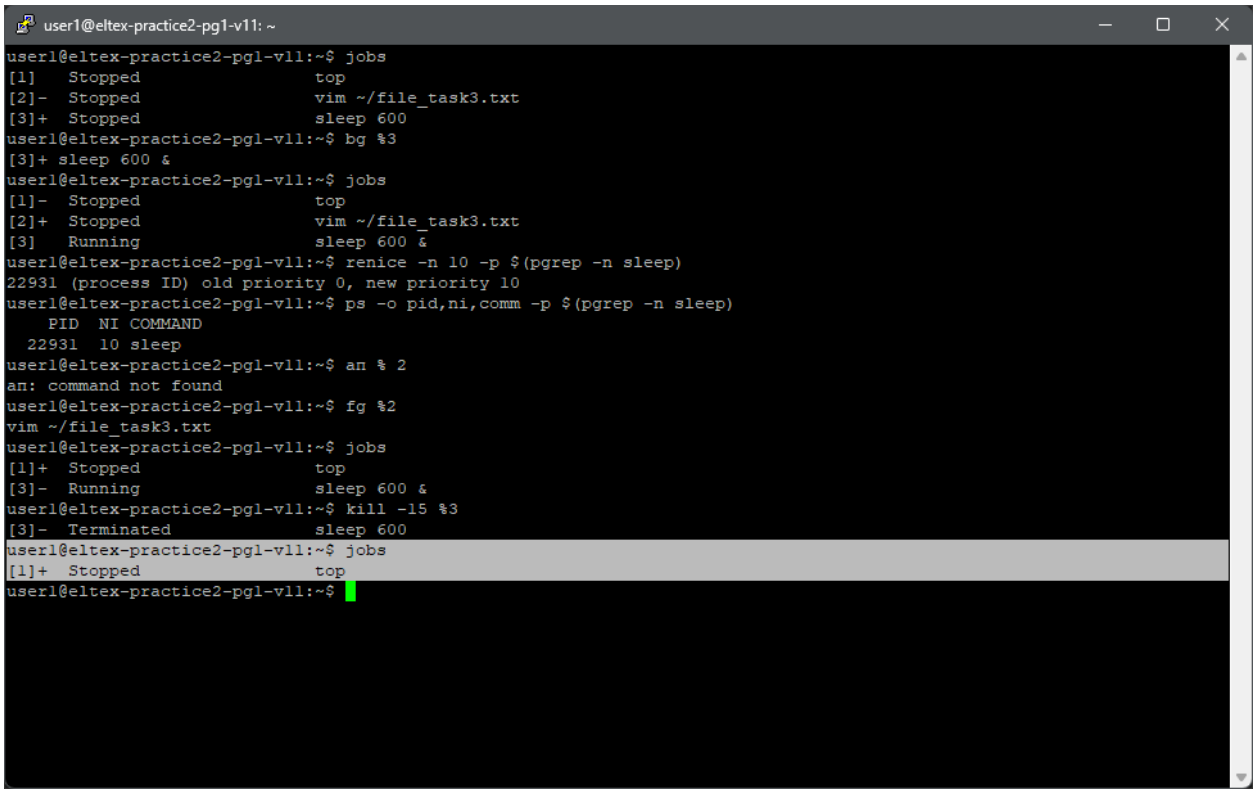


Выход из редактора vim

12)

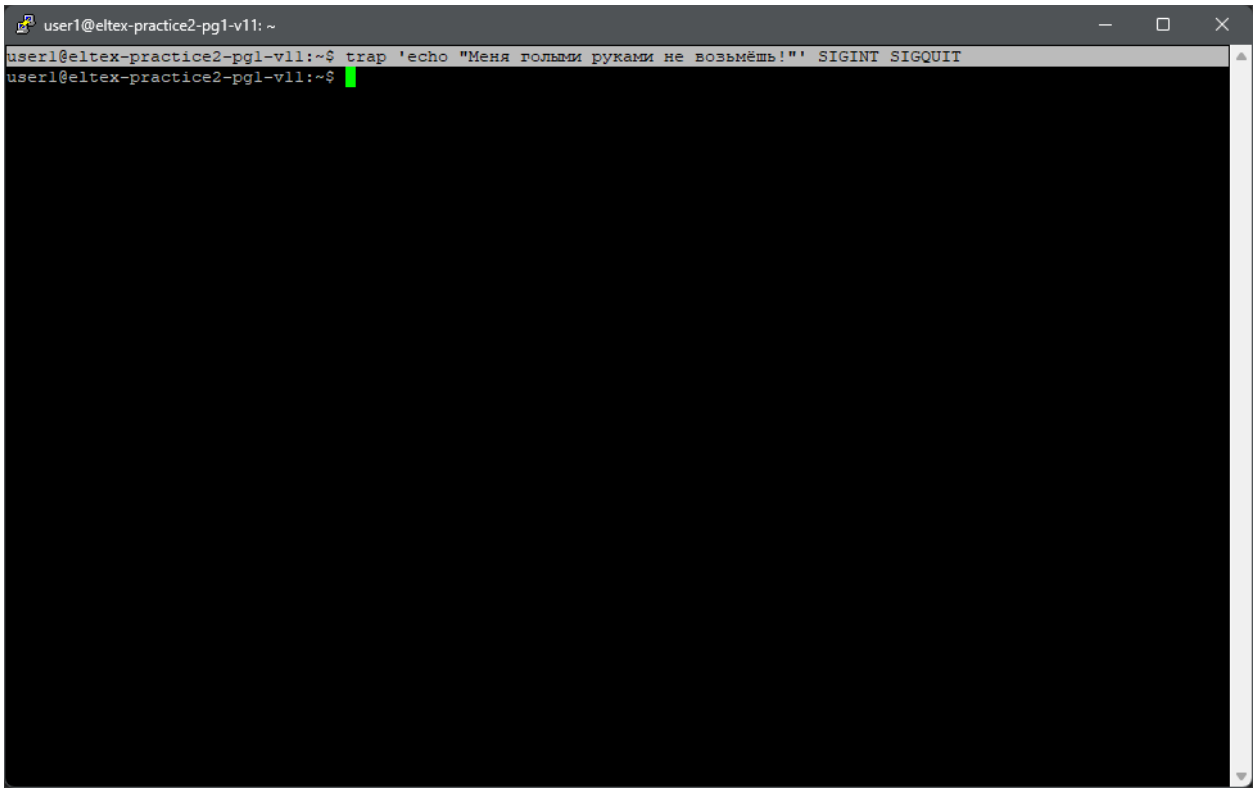


Отправка процессу sleep 600 сигнала SIGTERM

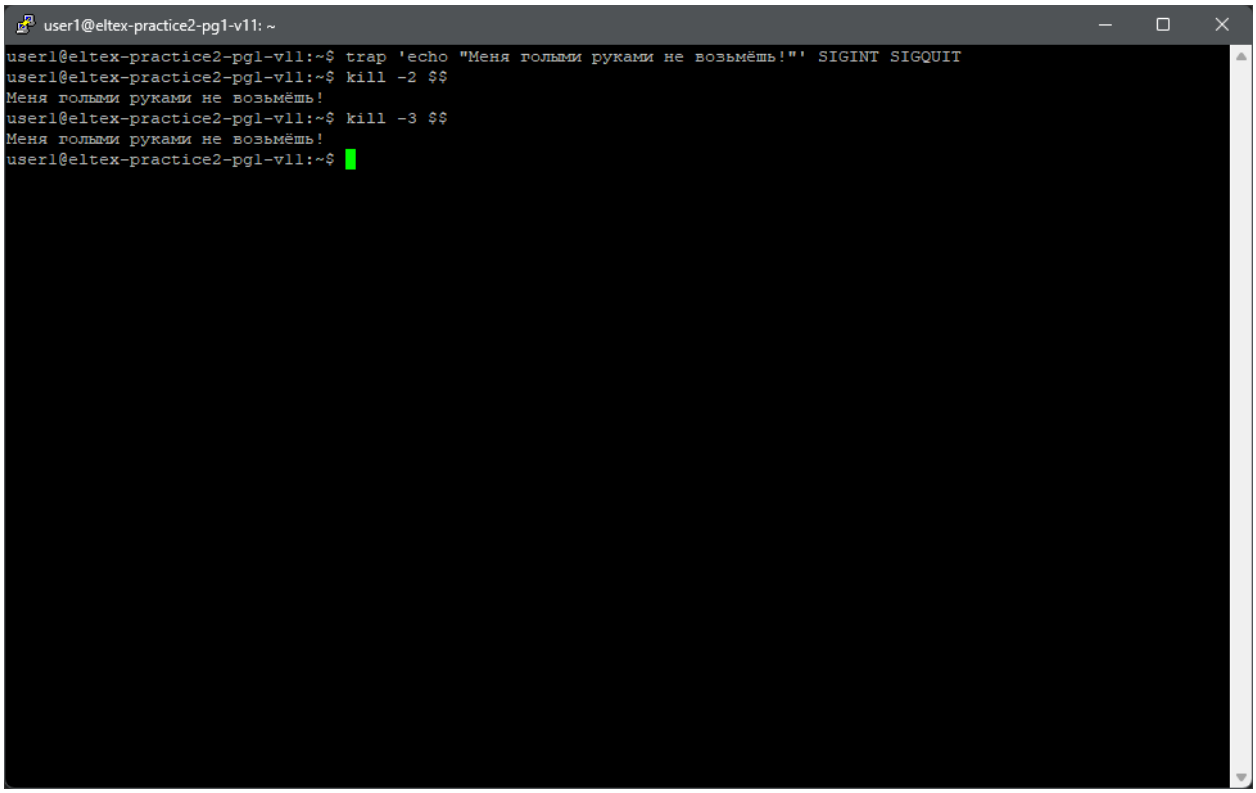


Проверка завершения процесса

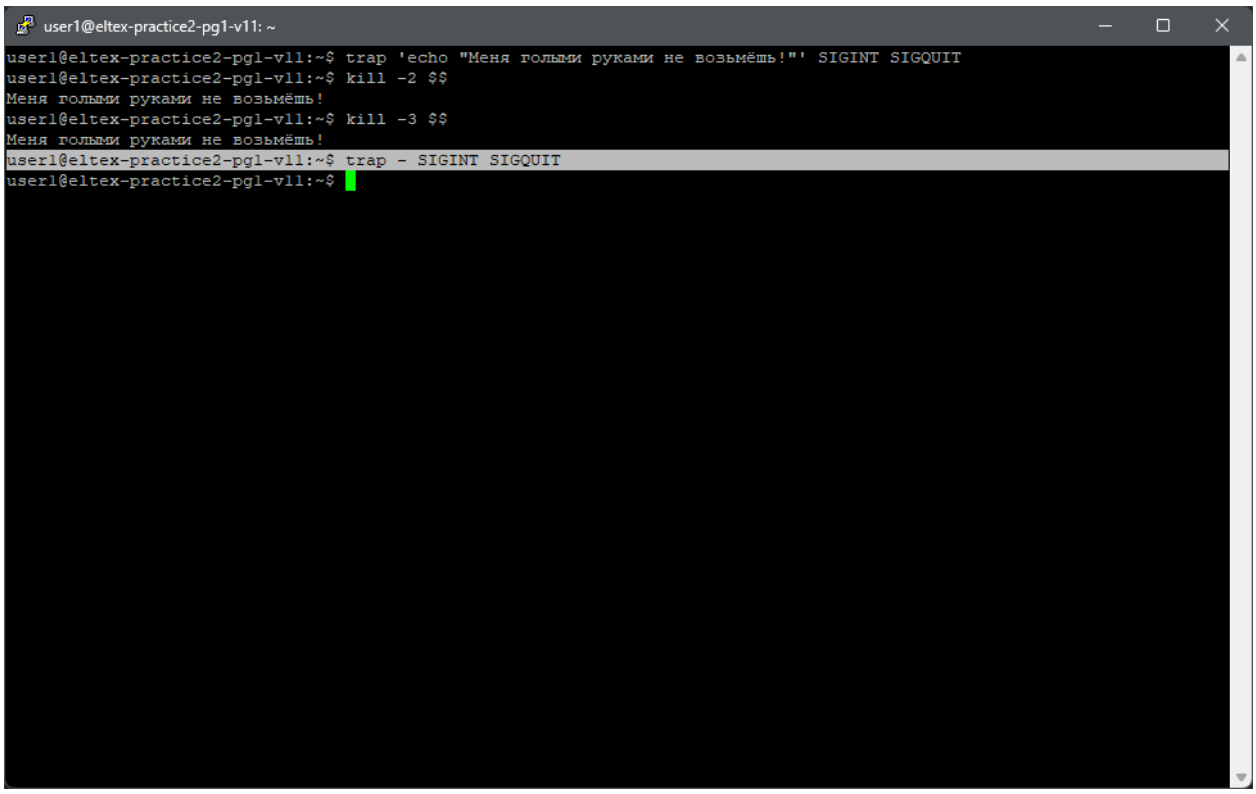
13)



Создание перехватчика сигналов

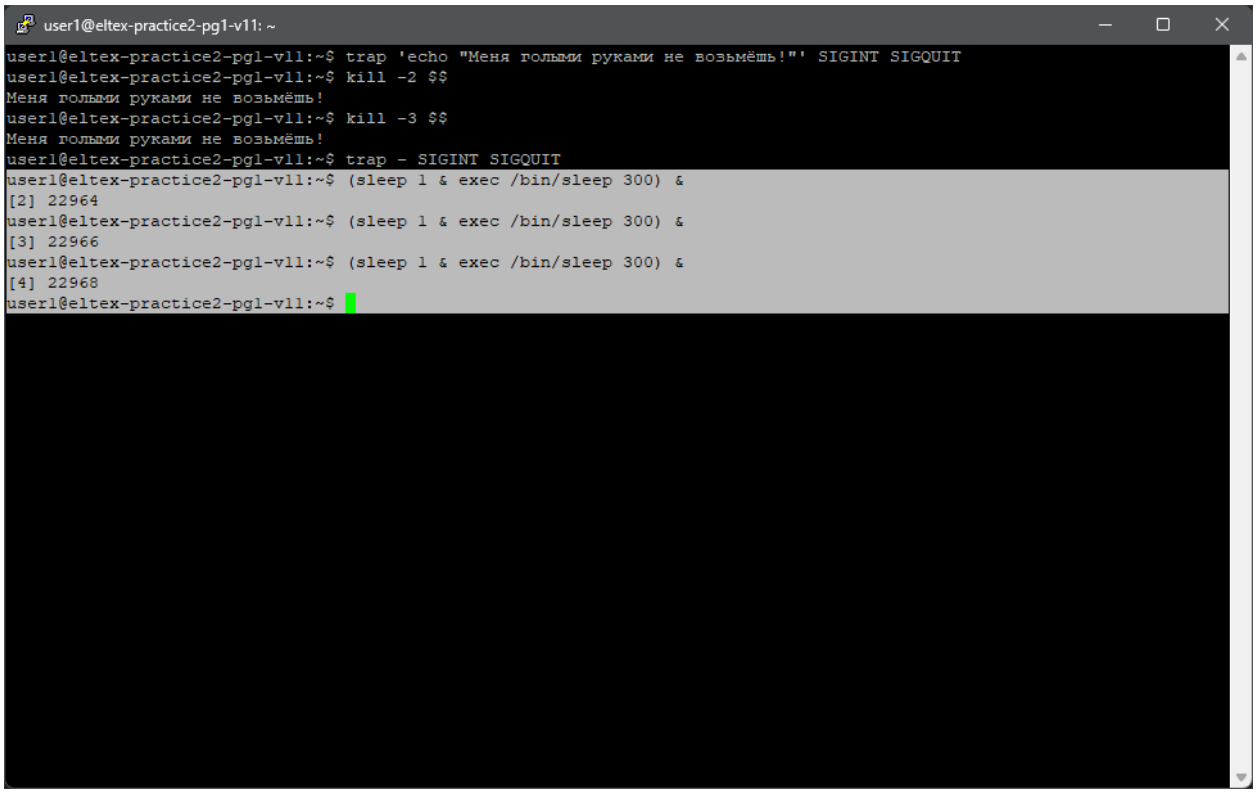


Проверка работы перехватчика



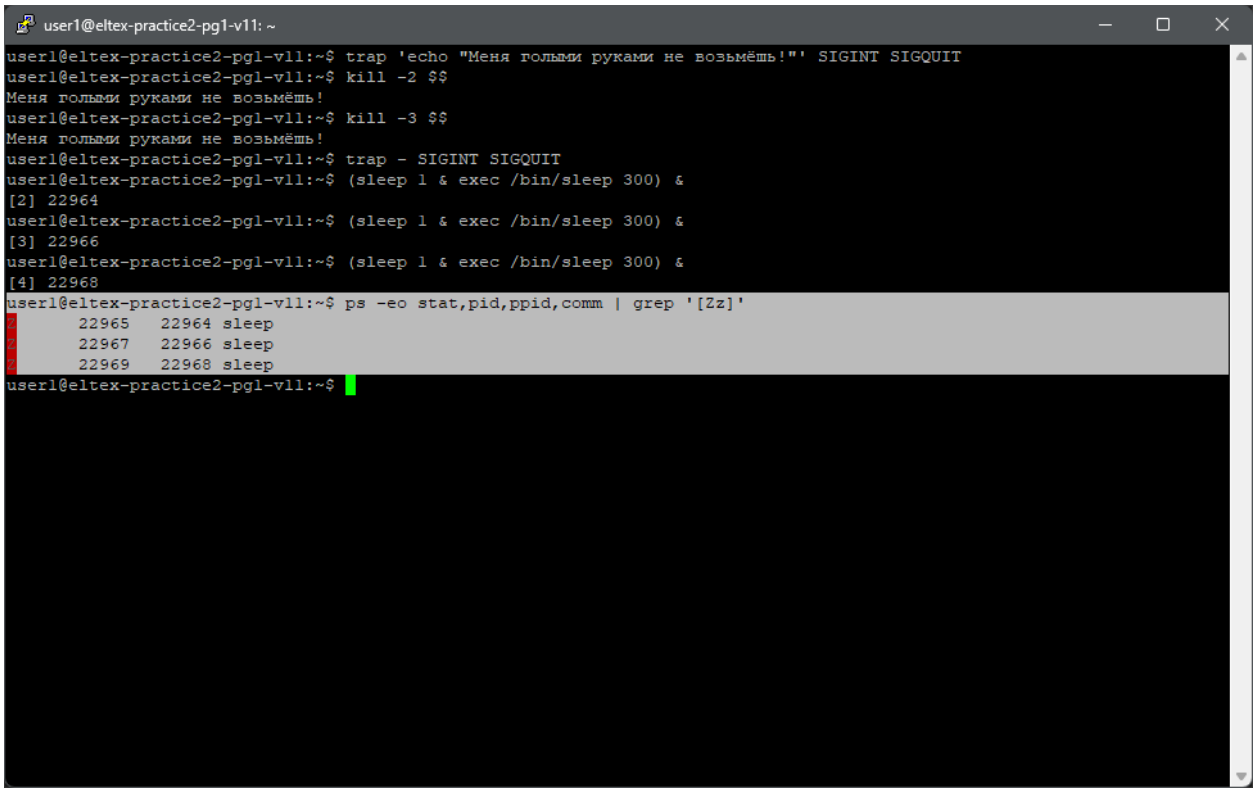
Отключение перехватчика

14)

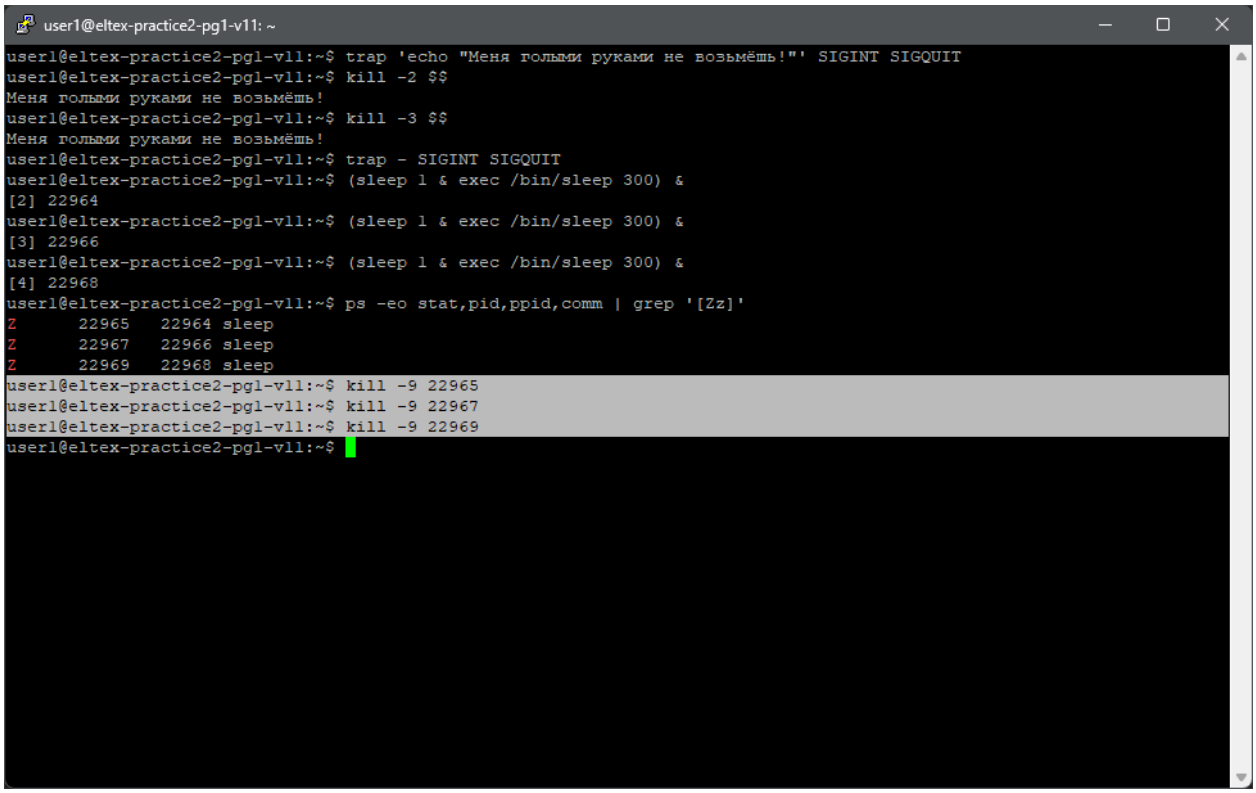


Создание зомби процессов

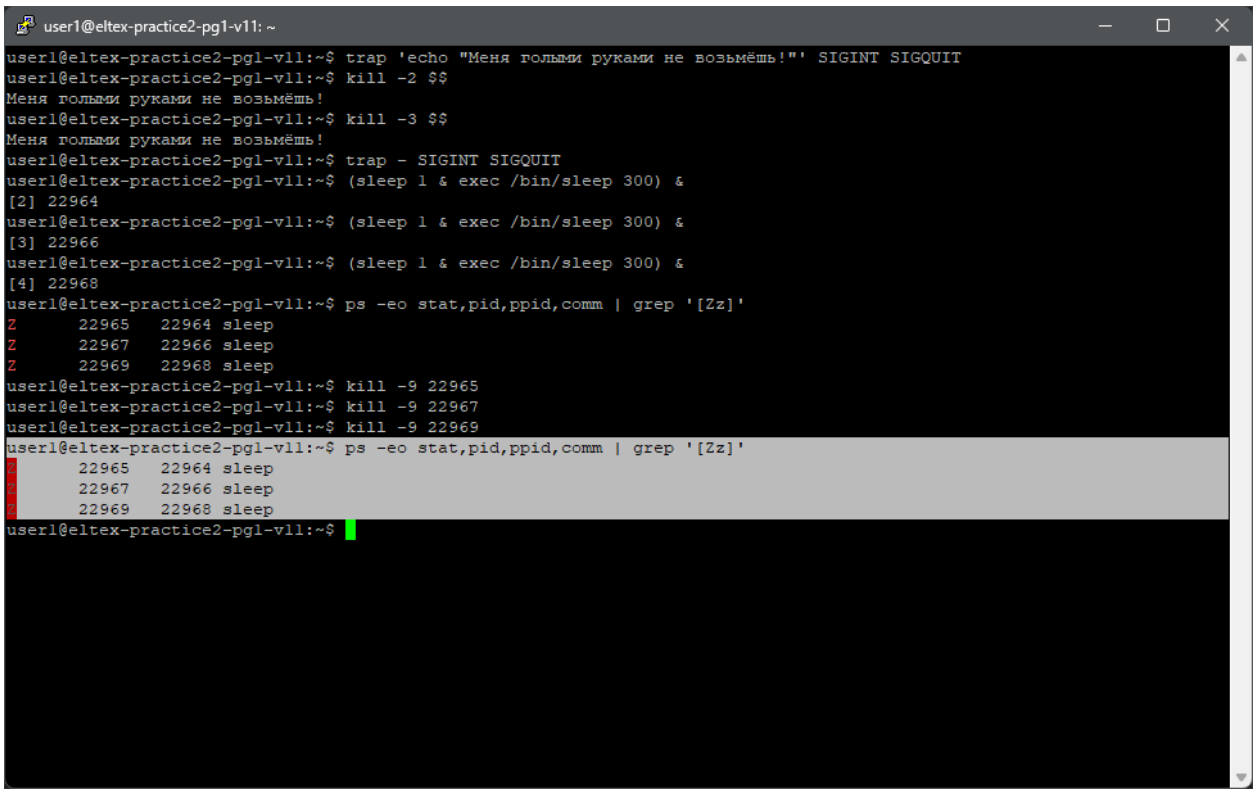
15)



Вывод запущенных зомби процессов



Попытка завершения зомби процессов



Проверка завершения зомби процессов

В операционных системах семейства Linux процесс переходит в состояние зомби (обозначается статусом Z или пометкой <defunct>), когда он завершил свое выполнение, но его родительский процесс еще не прочитал его код возврата с помощью системного вызова wait() или waitpid(). В таком состоянии зомби-процесс не потребляет вычислительных ресурсов процессора и не занимает оперативную память, освобождая свое адресное пространство. Однако запись о нем продолжает удерживаться в системной таблице дескрипторов процессов ядра ОС для сохранения идентификатора PID и статуса выхода. Уничтожить зомби-процесс напрямую с помощью сигналов (включая SIGKILL / kill -9) принципиально невозможно, так как процесс уже является мертвым и не способен обрабатывать сигналы ядра. Единственным способом устранения зомби является корректное считывание кода возврата родительским процессом. Если родительский процесс завис или не вызывает wait(), зомби можно удалить только путем завершения самого родительского процесса (PPID). В этом случае процесс-зомби становится «сиротой» и автоматически усыновляется корневым процессом инициализации системы (PID=1, systemd или init). Корневой процесс периодически вызывает системную функцию wait() для всех усыновленных задач, мгновенно освобождая дескрипторы зомби из таблицы ядра. Большое скопление неудаленных зомби опасно тем, что может полностью исчерпать доступный пул идентификаторов PID в системе, заблокировав порождение любых новых процессов. Таким образом, предотвращение появления зомби лежит исключительно на разработчиках программного обеспечения в части корректной обработки завершения дочерних потоков.